THIRSTY'S CODEX

STANDARDS EDITION

VOLUME XIII / 13 OF 20

REFERENCE REGISTERS AND WORKING DOCUMENTATION

Indexes, cross-references, source registers, checkpoints, and operational guides

NO PROOF, NO CLAIM

CONTINUOUS MASTER FOLIO / SOURCE-PRESERVING EDITION

Edition Statement

THIRSTY'S CODEX / STANDARDS EDITION / SOURCE-PRESERVING SET

This volume is one part of a continuous 20-volume edition. Canonical lifecycle section numbers remain stable across the set; no volume restarts or renumbers them.

The visual treatment has been rebuilt as a unified standards library: coordinated covers, volume architecture, source dividers, running headers, continuous master folios, local page numbers, and source-identity marks.

The source corpus remains authoritative over the presentation layer. Status terms are preserved exactly. CONTENT-COMPLETE does not silently become source-reconciled, hostile-reviewed, owner-accepted, ratified, or published.

Every registered source witness is identified by path, byte count, and SHA-256 digest. The complete source archive is preserved as an embedded attachment in Volume XX, while the two supplied foundational PDFs are preserved as attachments in Volume I.

CONTINUITY RULE

Read the master folio and canonical section sequence as one uninterrupted work. Each succeeding volume begins where the preceding volume ends.

Standards Edition Set Map

VOLUME I **Foundations and Governance**
Doctrine, prompts, front matter, shared governance, roles, and output contracts

VOLUME II **Research, Strategy, and Risk**
Lifecycle Sections 001-018

VOLUME III **Requirements through Hardware Architecture**
Lifecycle Sections 019-060

VOLUME IV **Electrical Engineering through User Validation**
Lifecycle Sections 061-106

VOLUME V **Software Systems through Smart Contracts**
Lifecycle Sections 107-150

VOLUME VI **Data, Intelligence, and Human Oversight**
Lifecycle Sections 151-188

VOLUME VII **Cybersecurity through Privacy Engineering**
Lifecycle Sections 189-218

VOLUME VIII **Law, Assurance, and Validation**
Lifecycle Sections 219-257

VOLUME IX **Configuration through Delivery Infrastructure**
Lifecycle Sections 258-300

VOLUME X **Production Operations and Command**
Lifecycle Sections 301-332

VOLUME XI **Support, Enterprise, and Roadmaps**
Lifecycle Sections 333-373

VOLUME XII **Maintenance, Decommissioning, and Retirement**
Lifecycle Sections 374-406

VOLUME XIII **Reference Registers and Working Documentation**
Indexes, cross-references, source registers, checkpoints, and operational guides

VOLUME XIV **Standards Source Library**
Preserved standards, command editions, blueprints, and source artifacts

VOLUME XV **Canonical Authority and Assurance**
Authority, constitutions, policies, schemas, provenance, evaluation, and verifica...

VOLUME XVI **Applied Corpus and V3Q**
Agent-output corpus, V3Q implementation, fixtures, tests, and governed examples

VOLUME XVII **Training Views and Indexes**
Continuous-pretraining, evaluation, retrieval, and training views with canonical ...

VOLUME XVIII **Agents and Owner Command Standards**
Agents 101 and Owner's Cockpit source-standard facsimiles

VOLUME XIX **Static Analysis and Tooling Evidence**
Codacy configuration, rulesets, analysis logs, and repository tooling

Volume Contents

486	Glossary	5
487	Index	16
488	Cross Reference Register	27
489	Source Register	33
490	Integration Checkpoint 001 018	35
491	Remaining Section Integration Roadmap	38
503	Agi Human Working Agreement	43
504	Batch Ledger	46
505	Codex Completion Path	48
506	Content Complete Status Transition	51
507	Readme	52
508	Component Relationships	53
509	High Level Architecture	56
510	Processing Pipeline	58
511	Project Summary	60
512	Project Summary	68



GLOSSARY

CODEX/90_REFERENCE/001_GLOSSARY.txt

SHA-256 d1587b5139bd28c7fda9e02bf0564ca1df6e07ac0bbc9f967cefa644907f1126

28,462 BYTES / PRESERVED SOURCE IDENTITY

GLOSSARY — WORKING DRAFT

Action contract
A durable record translating an intended consequential action into explicit objective, actor, authority, environment, scope, preconditions, risk, approval, state, and evidence requirements.

Agent
An artificial-intelligence process acting through a stable or attributable identity, runtime, role contract, capability set, assignment, and authority boundary.

Agent governance
The rules, authority boundaries, evidence requirements, verification gates, continuity practices, and enforcement mechanisms controlling how an agent may act and how its work is judged.

Agent role
A bounded professional function with defined inputs, responsibilities, prohibited behavior, outputs, and completion conditions.

Assumption
A proposition temporarily used for reasoning or planning without sufficient evidence to call it established.

Authority
The legitimate permission to make a decision or perform an action within a defined scope and consequence boundary. Authority may be bounded, delegated, expired, reduced, revoked, or exercised; it may not be invented.

Blocked
A state in which a prerequisite prevents the affected work from advancing safely or truthfully. Blocked is not the same as failed.

Capability
What an actor or system can technically do. Capability does not itself confer authority.

Codex section
One numbered lifecycle field in Thirsty's Codex, expressed as a governed work contract with applicability, responsibilities, artifacts, evidence, failure, recovery, and completion requirements.

Continuity map
A durable operational handoff record describing current state, completed work, evidence, failures, blockers, risks, unresolved conditions, and next safe action.

Constitutional work package
A pre-execution package containing objective, scope, source state, claims, assumptions, unknowns, obligations, roles, experiments, evidence contracts, acceptance criteria, rollback, and reversal conditions.

Cockpit
The governed human-system operating layer through which an accountable human understands state, exercises authority, directs work, inspects evidence, recovers, and stops or exits the system.

Delegation
A recorded grant of bounded authority from an authorized actor to another actor. Delegation narrows or transfers exercisable authority under conditions; it does not create new authority.

Dissent ledger
A durable record preserving material disagreement, its source, impact, disposition, decision owner, and reopening condition.

Evidence
An artifact, observation, command output, test result, log, configuration, trace, measurement, record, source, signature, hash, or other admissible basis supporting a bounded claim.

Critical uncertainty
A condition that is both materially consequential to a decision and genuinely unresolved, making it suitable for explicit scenario or sensitivity analysis.

Driver
A force, condition, dependency, or structural pressure capable of influencing future states over the relevant horizon.

Foresight
The governed examination of plausible change, uncertainty, alternative futures, consequences, indicators, and reversal conditions to improve present decisions without pretending to predict one inevitable future.

Future stakeholder
A person, community, owner, operator, institution, intelligent system, or other affected party that may inherit material consequences of a present decision.

Horizon
A defined future time boundary, lifecycle stage, or decision event used to bound analysis.

Indicator
An observable condition monitored for evidence that a scenario assumption, risk, opportunity, or decision basis is strengthening, weakening, or changing.

Path dependency
A condition in which earlier choices constrain later options, often increasing the cost or difficulty of change.

Point of no return
A decision or state boundary after which reversal becomes impossible, prohibited, or materially more costly or harmful.

Robust option
An option that remains useful or acceptable across multiple materially different plausible futures.

Scenario
A structured, internally traceable plausible future used to test decisions and consequences. A scenario is not a prediction.

Signal
A dated observation that may indicate emerging change but does not by itself establish a trend or future outcome.

Fail closed
A control posture in which missing authority, evidence, identity, integrity, or another required condition prevents the affected action rather than silently permitting it.

Human authority
The retained authority of the authorized human to define objectives, approve material decisions, accept risk, correct intent, revoke delegation, and make final determinations where human judgment is required.

Human-AI interaction
The governed exchange through which humans and AI systems communicate intent, questions, explanations, recommendations, plans, actions, evidence, corrections, uncertainty, and decisions.

Independent verification
Assessment of a claim by a role or mechanism not relying solely on the claimant's interpretation or the implementation under review.

Objective substitution
The unauthorized replacement, broadening, narrowing, sanitization, or displacement of the human's actual objective.

Operating contract
The standing behavioral and governance requirements that apply before a role-specific assignment begins.

Proof-carrying work product
A post-execution artifact set binding the delivered result to identity, changes, execution records, evidence, tests, hostile review, dissent, unresolved conditions, transition recommendation, rollback, and continuity.

Recovery

The governed process of returning from failure, degradation, error, or invalid state to a known safe state or an explicit human decision point.

Role contract
The source-defined professional boundary governing an assigned agent role.

Scope
The declared boundary of included work, exclusions, preservation requirements, acceptance criteria, stop conditions, and consequence limits.

Source identity
The information required to know what a source is, where it came from, which version or revision is being used, how current it is, and whether its integrity and authority are adequate for a claim.

Task and authority envelope
The assignment record specifying mode, objective, role, allowed tools, permissions, data boundaries, dependencies, resource limits, prohibited actions, acceptance criteria, evidence, stop conditions, and escalation.

Trusted transition
A controlled change in trusted state, such as promotion, quarantine, hold, rejection, or revocation, that cannot outrun authority and evidence.

Unknown
A material fact or state that has not been established. Unknown is information and must not be silently converted to false, zero, safe, or irrelevant.

Verification
The process of bounding a claim, checking identity and environment, examining evidence integrity, reproducing or falsifying where practical, and stating remaining uncertainty.

Vibe coder
A person who uses natural-language and AI-assisted creation to express and build an idea, including a person who may not know every professional discipline required to make the result real.

EXPANDED TERMS — SECTIONS 003–018

Anti-metric
An indicator used to detect when apparent success in a target metric is accompanied by displacement, gaming, exclusion, harm, degraded quality, or loss of the intended outcome.

Business rule
A source-bound statement defining a condition, decision, obligation, permission, prohibition, calculation, classification, or action within an organizational or domain context, including its owner, exceptions, jurisdiction, and effective period.

Business strategy
A coherent set of human-authorized choices about where to participate, whom to serve, how value will be created and sustained, which capabilities and dependencies are required, and what will not be pursued.

Bounded context
An explicit semantic and responsibility boundary inside which a domain model, terminology, rules, ownership, and source authority are internally coherent. Meaning crossing the boundary requires a defined contract or translation.

Capability model
A representation of what outcomes an organization, product, system, actor, or service must be able to produce, separated from the features, technologies, implementations, or providers used to realize those capabilities.

Commitment
An attributable promise or obligation with authorized speaker, exact beneficiary or recipient, scope, conditions, evidence, due state, change authority, and closure or breach handling.

Competitive intelligence

Lawful, ethical, source-grounded analysis of competitors, substitutes, ecosystem actors, alternatives, and relevant signals for a defined decision, with claimed and verified capability kept distinct.

Domain
The real-world or institutional field of meaning, behavior, rules, decisions, actors, information, authority, and lifecycle in which a system or change operates.

Domain event
A source-meaningful occurrence in the domain that has happened and may affect state, obligations, decisions, evidence, or downstream behavior. It is not merely a transport message.

Domain invariant
A condition that must remain true within a domain or bounded context. If a proposed state or action violates it, the transition is invalid regardless of aggregate benefit or confidence.

Entity
A domain concept whose identity and continuity matter across changes in attributes or state.

Innovation hypothesis
A bounded proposition that a proposed change can improve an evidenced outcome under declared conditions, with assumptions, failure hypotheses, experiment, evidence threshold, stop conditions, and reversal path.

Market research
Methodologically bounded inquiry into an external population, market, category, behavior, need, channel, price or funding condition, adoption barrier, or market structure for a defined decision.

Mental model
A person's internal understanding of how a system, task, concept, or process works, including expectations about available actions, consequence, evidence, failure, and recovery.

Mission invariant
A protected value, condition, prohibition, or decision boundary that downstream strategy and implementation may not silently trade away.

Mission strategy
The durable, human-ratified purpose, beneficiaries, protected values, non-goals, decision principles, outcome evidence, drift controls, and amendment path governing downstream choices.

Participant
A person who knowingly takes part in a research activity under the applicable consent, privacy, safety, accessibility, data-use, withdrawal, and grievance conditions.

Portfolio
The governed collection of initiatives, products, programs, projects, experiments, capabilities, maintenance obligations, and retirements competing for shared resources and carrying shared dependencies or risks.

Product management
The continuous decision function governing product discovery, intake, prioritization, roadmap, requirements, release, learning, support, change, deprecation, and retirement.

Product strategy
The evidence-grounded definition of what product should exist, for whom, to produce which outcomes, under which principles, capabilities, non-goals, lifecycle boundaries, and decision conditions.

Program
A coordinated group of related projects, products, workstreams, and changes managed together because the authorized outcome, integration, transition, or benefits cannot be achieved by one project alone.

Project
A temporary, bounded, authorized endeavor that creates or changes defined deliverables and outcomes under explicit scope, resources, dependencies, risk, evidence, acceptance, and closure conditions.

Source of truth

The authoritative system, record, artifact, institution, or owner from which a particular fact, state, definition, rule, or decision must be read. A source of truth is scoped; no source is automatically authoritative for every claim.

Stakeholder
A person, group, institution, owner, operator, user, affected non-user, partner, regulator, community, or other legitimate party whose authority, rights, interests, obligations, knowledge, benefit, or exposure materially affects the work.

State transition
A governed change from one defined domain or system state to another, including trigger, actor, authority, preconditions, guards, evidence, effects, failure handling, and resulting state.

User job
The outcome a user, owner, operator, or other actor is trying to achieve in a particular context, including the evidence needed before acting and the consequence of success, failure, or delay.

User journey
The end-to-end path from trigger and entry through information, decisions, actions, evidence, consequence, failure, recovery, exit, and continuity for a user or owner job.

User research
Governed inquiry using actual participants, observations, artifacts, and contextual evidence to understand human goals, behavior, language, mental models, accessibility, trust, failure, and recovery for a defined decision.

Value object
A domain concept defined primarily by its attributes and meaning rather than a persistent independent identity. Equality and replacement rules must be domain-defined.

EXPANDED TERMS — SECTIONS 017–018

Control effectiveness
The evidence-supported degree to which an implemented control achieves its objective in the declared environment, version, scope, and period. Documentation or configuration presence alone does not establish effectiveness.

Disqualifying condition
A mandatory constraint violation or other condition that makes an option ineligible within the declared scope, regardless of its aggregate score or benefits.

Feasibility
The evidence-bounded ability to realize, operate, govern, support, recover, maintain, and retire an option under declared authority, constraints, resources, dependencies, environment, scale, and lifecycle conditions.

Feasibility condition
A requirement, dependency, approval, resource, control, evidence threshold, environment, or operating limitation that must remain true for a feasibility determination to hold.

Inherent risk
Risk assessed before considering the effect of current controls.

Risk
Uncertainty that may affect an objective, value, right, person, system, commitment, or environment. A useful risk statement identifies a source or cause, an uncertain event, and a consequence within a bounded context.

Risk acceptance
An attributable, authorized, evidence-informed decision to tolerate a specifically bounded residual risk for a defined scope, period, and set of conditions. Continued work or silence is not acceptance.

Risk appetite
The human-authorized type and amount of risk an accountable entity is prepared to pursue or retain in support of objectives, subject to mandatory legal, safety, rights, privacy, security, and other constraints.

Risk tolerance

A defined permissible variation or threshold around an objective or risk appetite, including the point at which escalation, treatment, degradation, or halt is required.

Residual risk
Risk remaining after considering the verified effect of current or proposed controls. Current residual risk and projected residual risk after planned treatment must be kept distinct.

Safe halt
A governed state that prevents new prohibited or consequential actions while preserving authorized diagnosis, evidence, communication, and recovery pathways.

Treatment-created risk
New or increased risk introduced by a proposed risk treatment, such as added complexity, surveillance, lock-in, operator burden, control conflict, or recovery fragility.

EXPANDED TERMS — SECTION 019

Acceptance criterion
An observable condition, threshold, state, journey outcome, or evidence requirement used to determine whether a requirement has been satisfied within its declared scope and environment.

Derived requirement
A requirement produced through documented reasoning from a higher-level objective, need, constraint, risk treatment, architecture obligation, or governing source. Its derivation and authority must remain traceable.

Functional requirement
A bounded statement of behavior, service, response, transformation, or interaction the system or designated actor must perform under declared conditions.

Nonfunctional requirement
A requirement concerning a quality, constraint, operating condition, or cross-cutting property such as security, privacy, safety, accessibility, reliability, performance, maintainability, recoverability, or governance. The label does not make it optional.

Requirement
A versioned, attributable, bounded, and verifiable obligation or constraint that states what outcome, behavior, quality, control, evidence, or lifecycle condition must be satisfied within declared applicability.

Requirement baseline
An attributable, approved, immutable or change-controlled version of the in-scope requirement set, including scope, variants, open findings, source versions, approvals, and effective date.

Requirement criticality
The consequence-based importance of satisfying a requirement, distinct from implementation priority or scheduling order.

Requirement source
The authoritative artifact, law, policy, contract, decision, stakeholder statement, user evidence, domain rule, feasibility condition, risk treatment, or other traceable origin supporting a requirement.

Requirements traceability
The durable, bidirectional relationships connecting sources, needs, requirements, architecture, implementation, tests, evidence, acceptance, operation, change, and retirement.

Requirements validation
Evaluation of whether the requirement set expresses the authorized objective and legitimate stakeholder, user, domain, lifecycle, and governing needs for the declared context.

Requirements verification
Evaluation of whether requirement statements and their management records meet defined quality, consistency, traceability, structure, and evidence rules before implementation satisfaction is assessed.

EXPANDED TERMS — SECTION 020

Concept of operations
A controlled description of how a system is intended to be used, supervised, supported, degraded, recovered, changed, and retired by identified actors in declared environments and lifecycle conditions.

Configuration item
A system element, artifact, model, interface, data set, policy, environment, procedure, or other controlled object whose identity, version, status, relationships, and change history must be managed.

Enabling system
A system, service, facility, process, organization, tool, supplier, or capability that supports development, integration, verification, validation, transition, operation, maintenance, or retirement of the system of interest without necessarily being part of the delivered system.

Interface control
The governed definition and change management of a boundary between producers and consumers, including exchanged data or resources, protocol, timing, units, identity, authority, errors, compatibility, security, evidence, and ownership.

Measure of effectiveness
A measure of whether the system produces the authorized mission, owner, user, or operational outcome in the intended context.

Measure of performance
A measure of how well a system or system element performs a defined function under declared conditions.

System baseline
An attributable, versioned, approved, and change-controlled representation of the system at a defined lifecycle point, including applicable requirements, elements, interfaces, configuration, evidence, open findings, and authority.

System element
A bounded part of a system with defined purpose, responsibilities, inputs, outputs, dependencies, authority, interfaces, failure behavior, recovery behavior, ownership, and verification method.

System of interest
The system selected as the primary subject of analysis, design, implementation, integration, verification, validation, operation, maintenance, or retirement, with an explicit boundary separating it from external and enabling systems.

Systems Engineering
The interdisciplinary lifecycle function that converts an approved requirements baseline into a coherent whole-system definition, allocation, interface model, technical baseline, integration path, verification and validation strategy, operational transition, support model, and retirement path.

Technical performance measure
A tracked technical characteristic, margin, or threshold used to assess whether the evolving system is likely to satisfy a critical requirement or system objective.

Validation
Evidence-based determination that the integrated system solves the right problem and supports the intended outcomes and complete journeys in the declared operational context.

Verification
Evidence-based determination that a specified requirement, interface, element, or system claim has been satisfied within its declared scope, configuration, environment, and method.

EXPANDED TERMS — SECTIONS 021–022

Architecture decision record
A durable record of a material architecture decision containing context, decision owner, authority state, options, criteria, evidence, assumptions, tradeoffs, selected or recommended outcome, dissent, conditions, consequences, review trigger, and supersession history.

Architecture principle
A human-authorized rule or strong preference guiding architecture decisions across a declared scope. A principle must identify its authority, rationale, applicability, exceptions, and review condition.

Architecture roadmap
A dependency-aware sequence of current, transition, and target states with decisions, increments, evidence gates, migration, rollback, support, and retirement obligations.

Capability
An outcome-producing ability that an enterprise, system, organization, product, service, human, or agent must be able to sustain. A capability is distinct from the technology, team, feature, or provider used to realize it.

Current-state architecture
A source-bound representation of the architecture that actually exists for a declared version, environment, and evidence date. It must not include planned or inferred elements as though they were implemented.

Enterprise Architecture
The governed alignment of mission, strategy, operating model, capabilities, value streams, organizations, information, applications, technology, controls, investment, transition, and lifecycle across an enterprise boundary.

Exception
An authorized, bounded, time- or condition-limited departure from a principle, standard, requirement, or baseline, with rationale, risk, owner, evidence, expiry, monitoring, and closure or renewal path.

Reference architecture
A reusable architecture pattern defining expected layers, responsibilities, interfaces, constraints, and prohibited shortcuts for a class of solutions without claiming that a specific implementation exists.

Solution Architecture
The governed design of a bounded solution that maps requirements and context to components, interfaces, data, state, authority, security, deployment, operations, evidence, recovery, maintenance, and retirement.

Target-state architecture
An approved or advisory representation of a desired future architecture, explicitly distinguished from current and transition states and bound to assumptions, evidence, authority, and acceptance conditions.

Transition architecture
An intermediate architecture required to move safely from one controlled state to another while preserving continuity, compatibility, authority, evidence, recovery, and rollback.

Value stream
The end-to-end sequence through which an initiating need is transformed into an outcome for a stakeholder or affected party, including decisions, capabilities, processes, information, systems, evidence, support, and recovery.

EXPANDED TERMS — SECTIONS 023–024

Anti-corruption layer
A controlled translation boundary that prevents one domain context's model, language, assumptions, or authority from leaking directly into another context.

Architecture invariant
A condition that must remain true across architecture states, implementations, interfaces, and changes. An invariant is treated as a mandatory constraint rather than a preference that can be outweighed by aggregate scoring.

Bounded context
A declared boundary within which a domain model, language, rules, state, authority, and ownership have consistent meaning. Meaning crossing the boundary requires an explicit contract or translation.

Context map
A governed representation of bounded contexts and their relationships, including upstream and downstream responsibility, translation, shared models, published language, dependencies, authority, and lifecycle state.

Domain Architecture

The governed organization of domain boundaries, language, models, rules, state, decisions, data ownership, services, events, authority, integrations, and change across bounded contexts.

Domain service
A domain-owned function that performs behavior belonging to a bounded context when that behavior does not naturally belong to one entity or value object. Its authority and interfaces remain explicit.

Failure domain
A bounded set of system elements, state, dependencies, or operations that may fail together or share a common cause, together with the expected blast radius, containment, detection, and recovery behavior.

Published language
A versioned, owned contract through which one domain context exposes defined meaning to other contexts without exposing or imposing its entire internal model.

Semantic contract
A versioned agreement defining the meaning, source, authority, timing, validity, error, correction, and lifecycle semantics of information or behavior exchanged across a boundary.

State ownership
The explicit assignment of authoritative responsibility for creating, changing, validating, reconciling, evidencing, and retiring a defined state.

Systems Architecture
The governed structural architecture of a system of interest, defining its elements, responsibilities, interfaces, dependencies, state, data, authority, trust, evidence, failure, recovery, deployment, operation, maintenance, and retirement.

EXPANDED TERMS — SECTIONS 025–026

Business Architecture
The governed structure connecting mission and strategy to value propositions, outcomes, value streams, capabilities, organizations, roles, decisions, policies, information, products, services, resources, measures, investments, controls, transitions, and lifecycle.

Business capability
An enduring outcome-producing ability required by an enterprise or bounded organization, independent of the temporary organization, project, process, technology, provider, or product used to realize it.

Capability Architecture
The governed definition of required capabilities, including their outcomes, boundaries, ownership, realization, dependencies, levels, gaps, measures, evidence, resilience, increments, investments, transitions, and lifecycle state.

Capability catalog
A versioned register of capabilities and their definitions, owners, outcomes, scope, realization, dependencies, measures, evidence, risks, controls, and lifecycle states.

Capability increment
A bounded, dependency-aware change intended to establish or improve a defined capability outcome under explicit authority, acceptance criteria, evidence gates, transition, support, and rollback conditions.

Capability map
A governed structural view of capabilities and their relationships, decomposition, ownership, dependencies, and lifecycle state. A map does not by itself prove capability existence or effectiveness.

Capability realization
The combination of people, roles, competencies, processes, functions, information, data, applications, technologies, facilities, partners, suppliers, AI, controls, and governance through which a capability is delivered.

Capability state
The truthful status of a capability within a declared scope and environment, such as required, proposed, existing, available, authorized, demonstrated, operational, effective, degraded, unsupported, superseded, or retired.

Operating model
The governed arrangement through which value streams, capabilities, organizations, roles, decision rights, processes, information, technology, locations, partners, resources, and controls combine to deliver outcomes.

Value proposition
A source-grounded statement of the outcome or benefit intended for a defined stakeholder or affected party, together with obligations, assumptions, harms to avoid, acceptance boundaries, and evidence methods.



INDEX

CODEX/90_REFERENCE/002_INDEX.txt

SHA-256 ffab5e637cb515b43de46922cc551bb85c0e6dcea63fc59877ea883e5205d9d3

20,412 BYTES / PRESERVED SOURCE IDENTITY

INDEX — SECTION TITLES

A/B Testing — Section 370
Abuse Prevention — Section 216
Acceptance Testing — Section 237
Accessibility Architecture — Section 053
Accessibility Design — Section 103
Accessibility Testing — Section 247
Account Closure — Section 396
Adaptive Maintenance — Section 377
Agent Systems Engineering — Section 178
AI Architecture — Section 056
AI Engineering — Section 169
AI Governance — Section 183
AI Operations — Section 308
AI Safety — Section 182
Alerting — Section 318
Algorithm Engineering — Section 109
Analytics Engineering — Section 161
Animation Engineering — Section 141
API Architecture — Section 042
API Engineering — Section 128
Application Architecture — Section 032
Application Engineering — Section 117
Application Operations — Section 309
Application Security — Section 190
AR Design — Section 091
Artifact Management — Section 262
Assembly — Section 353
Audio Engineering — Section 142
Audit — Section 226
Authentication Engineering — Section 201
Authorization Engineering — Section 202
Availability Management — Section 322
Backend Engineering — Section 119
Backup Engineering — Section 328
Billing Engineering — Section 359
Blockchain Engineering — Section 149
Blue Teaming — Section 212
Brand Management — Section 345
Browser Engineering — Section 126
Build Automation — Section 265
Build Engineering — Section 264
Business Analysis — Section 011
Business Architecture — Section 025
Business Continuity — Section 326
Business Intelligence — Section 162
Business Strategy — Section 005
Capability Architecture — Section 026
Capacity Management — Section 321
Certificate Management — Section 298
Change Management — Section 289
Chaos Engineering — Section 245
Cloud Architecture — Section 036
Cloud Engineering — Section 277
Cloud Operations — Section 304

Cloud Security — Section 194
Command and Control — Section 332
Commerce — Section 358
Communications Architecture — Section 040
Community Management — Section 337
Compatibility Management — Section 387
Compatibility Testing — Section 248
Competitive Intelligence — Section 013
Compiler Engineering — Section 111
Compliance Architecture — Section 049
Compliance Testing — Section 251
Computer Engineering — Section 063
Computer Science — Section 108
Computer Vision — Section 172
Configuration Delivery — Section 297
Configuration Management — Section 258
Container Engineering — Section 279
Container Orchestration — Section 280
Content Architecture — Section 054
Content Delivery Networks — Section 300
Content Design — Section 097
Content Strategy — Section 098
Continuous Delivery — Section 267
Continuous Deployment — Section 268
Continuous Integration — Section 266
Control Systems Engineering — Section 073
Conversational Design — Section 089
Copywriting — Section 100
Corrective Maintenance — Section 375
Cross-Platform Engineering — Section 124
Cryptography — Section 203
Customer Analytics — Section 366
Customer Success — Section 336
Customer Support — Section 334
Cybersecurity Engineering — Section 189
Data Analysis — Section 163
Data Architecture — Section 031
Data Archival — Section 392
Data Deletion — Section 395
Data Engineering — Section 153
Data Governance — Section 155
Data Integration — Section 157
Data Lake Engineering — Section 160
Data Migration — Section 158
Data Migration Operations — Section 382
Data Modeling — Section 154
Data Operations — Section 307
Data Quality Engineering — Section 156
Data Retention — Section 391
Data Science — Section 164
Data Testing — Section 254
Data Warehousing — Section 159
Database Administration — Section 285
Database Engineering — Section 151
Deep Learning Engineering — Section 168
Dependency Management — Section 260

Dependency Updates — Section 381
Deployment Engineering — Section 290
Deprecation Management — Section 389
Design Systems — Section 104
Desktop Engineering — Section 122
Developer Experience — Section 273
Developer Relations — Section 340
Device Provisioning — Section 355
DevOps — Section 269
DevSecOps — Section 270
Digital Forensics — Section 213
Digital Preservation — Section 393
Digital Twin Engineering — Section 137
Disaster Recovery — Section 327
Distributed Systems Architecture — Section 038
Distributed Systems Engineering — Section 131
Distribution — Section 357
Distribution Engineering — Section 291
Documentation Engineering — Section 341
Domain Analysis — Section 016
Domain and DNS Management — Section 299
Domain Architecture — Section 024
Driver Engineering — Section 115
Edge Architecture — Section 037
Edge Computing Engineering — Section 278
Education — Section 339
Electrical Engineering — Section 061
Electronics Engineering — Section 062
Embedded Security — Section 198
Embedded Software Engineering — Section 069
Embedded Systems Architecture — Section 059
End-to-End Testing — Section 236
Endpoint Security — Section 196
Enterprise Architecture — Section 021
Environment Management — Section 263
Environment Promotion — Section 295
Environmental Disposal / Recycling — Section 401
Ethics — Section 222
Event-Driven Architecture — Section 043
Experience Design — Section 077
Experimentation — Section 369
Extension Engineering — Section 127
Failover Engineering — Section 330
Feasibility Analysis — Section 017
Feature Management — Section 296
Feedback Management — Section 372
Financial Operations — Section 363
Firmware Engineering — Section 068
Firmware Updates — Section 379
Foresight / Futures — Section 002
Fraud Prevention — Section 215
Frontend Engineering — Section 118
Full-Stack Engineering — Section 120
Functional Architecture — Section 028
Game Design — Section 094
Game Engineering — Section 138

Generative AI Engineering — Section 170
Geographic Information Systems — Section 148
GitOps — Section 271
Governance Architecture — Section 048
Graphic Design — Section 085
Graphics Engineering — Section 139
Growth Engineering — Section 368
Hardware Architecture — Section 060
Hardware Certification — Section 354
Hardware Disposal — Section 399
Hardware Security — Section 199
Hardware Testing — Section 232
Help Desk — Section 335
High-Performance Computing — Section 133
Human-in-the-Loop Operations — Section 188
Human–Computer Interaction — Section 075
Identity and Access Management — Section 200
Identity Architecture — Section 046
Incident Management — Section 319
Incident Response — Section 214
Industrial Design — Section 065
Information Architecture — Section 030
Infrastructure Architecture — Section 035
Infrastructure as Code — Section 276
Infrastructure Decommissioning — Section 398
Infrastructure Engineering — Section 275
Infrastructure Security — Section 193
Innovation — Section 003
Installation Engineering — Section 293
Integration Architecture — Section 041
Integration Engineering — Section 129
Integration Testing — Section 234
Intellectual Property — Section 223
Interaction Design — Section 076
Internal Controls — Section 227
Internationalization — Section 102
Internet of Things Engineering — Section 070
IT Operations — Section 312
Kernel Engineering — Section 114
Key Management — Section 204
Knowledge Architecture — Section 055
Knowledge Graph Engineering — Section 175
Knowledge Management — Section 342
Legal — Section 219
Lessons Learned — Section 404
Level Design — Section 095
Licensing — Section 224
Licensing Operations — Section 364
Lifecycle Governance — Section 405
Load Testing — Section 240
Localization — Section 101
Localization Testing — Section 250
Logging — Section 315
Logistics — Section 356
Machine Learning Architecture — Section 057
Machine Learning Engineering — Section 167

Maintenance Engineering — Section 374
Manufacturing — Section 351
Market Research — Section 012
Mechanical Engineering — Section 064
Media Engineering — Section 144
Metrics Engineering — Section 317
Microservices Engineering — Section 130
Middleware Engineering — Section 116
Mission Strategy — Section 004
MLOps — Section 184
Mobile Engineering — Section 123
Mobile Security — Section 197
Model Alignment — Section 181
Model Evaluation — Section 180
Model Operations — Section 185
Model Testing — Section 253
Model Training — Section 179
Modernization — Section 384
Monitoring — Section 314
Motion Design — Section 086
Narrative Design — Section 096
Native Application Engineering — Section 125
Natural Language Processing — Section 171
Network Architecture — Section 039
Network Engineering — Section 284
Network Operations — Section 305
Network Security — Section 195
Observability Engineering — Section 313
Ontology Engineering — Section 176
Operating Systems Engineering — Section 113
Operational Architecture — Section 029
Operational Readiness — Section 331
Operations Research — Section 166
Optimization — Section 371
Package Management — Section 261
Packaging — Section 292
Parallel Computing — Section 134
Payments Engineering — Section 360
PCB Design — Section 067
Penetration Testing — Section 210
Performance Management — Section 323
Performance Testing — Section 239
Platform Architecture — Section 034
Platform Engineering — Section 272
Platform Migration — Section 383
Platform Operations — Section 310
Platform Security — Section 192
Policy — Section 221
Portfolio Management — Section 007
Post-Implementation Review — Section 402
Postmortem Analysis — Section 403
Preventive Maintenance — Section 376
Privacy Architecture — Section 047
Privacy Engineering — Section 218
Problem Management — Section 320
Process Architecture — Section 027

Procurement — Section 348
Product Analytics — Section 367
Product Management — Section 010
Product Marketing — Section 343
Product Security — Section 191
Product Strategy — Section 006
Production Engineering — Section 302
Production Planning — Section 352
Program Management — Section 008
Programming Language Engineering — Section 110
Project Management — Section 009
Prompt Engineering — Section 186
Prototyping — Section 105
Provisioning — Section 294
Quality Assurance — Section 230
Quality Engineering — Section 229
Quality Management — Section 228
Real-Time Systems Engineering — Section 132
Recommendation Systems Engineering — Section 147
Records Disposal — Section 394
Records Management — Section 225
Red Teaming — Section 211
Refactoring — Section 385
Regression Testing — Section 238
Regulatory Compliance — Section 220
Reinforcement Learning — Section 174
Release Engineering — Section 287
Release Management — Section 288
Reliability Architecture — Section 051
Reliability Management — Section 324
Reliability Testing — Section 243
Rendering Engineering — Section 140
Requirements Engineering — Section 019
Research — Section 001
Resilience Architecture — Section 052
Resilience Testing — Section 244
Restore Engineering — Section 329
Revenue Operations — Section 362
Risk Management — Section 018
Roadmap Management — Section 373
Robotics Architecture — Section 058
Robotics Engineering — Section 074
Runtime Architecture — Section 044
Runtime Engineering — Section 112
Runtime Operations — Section 303
Safety Architecture — Section 050
Safety Testing — Section 252
Sales Engineering — Section 346
Scalability Testing — Section 242
Scientific Computing — Section 135
Search Engineering — Section 146
Secrets Management — Section 205
Secure Destruction — Section 400
Security Architecture — Section 045
Security Operations — Section 206
Security Operations — Section 306

Security Patching — Section 380
Security Testing — Section 246
Semantic Systems Engineering — Section 177
Semiconductor Engineering — Section 066
Server Engineering — Section 282
Service Decommissioning — Section 397
Service Design — Section 078
Service Management — Section 311
Service-Level Management — Section 325
Signal Processing — Section 072
Simulation Engineering — Section 136
Site Reliability Engineering — Section 301
Smart Contract Engineering — Section 150
Software Architecture — Section 033
Software Engineering — Section 107
Software Testing — Section 231
Software Updates — Section 378
Solution Architecture — Section 022
Solutions Engineering — Section 347
Sound Design — Section 087
Source Control Management — Section 259
Spatial Interface Design — Section 090
Speech Engineering — Section 173
Stakeholder Management — Section 014
Statistics — Section 165
Storage Administration — Section 286
Storage Engineering — Section 152
Streaming Engineering — Section 145
Stress Testing — Section 241
Subscription Management — Section 361
Sunset Planning — Section 390
Supply Chain Management — Section 350
Synthetic Data Engineering — Section 187
System Retirement — Section 406
System Testing — Section 235
Systems Administration — Section 283
Systems Architecture — Section 023
Systems Engineering — Section 020
Technical Debt Management — Section 386
Technical Marketing — Section 344
Technical Support — Section 333
Technical Writing — Section 099
Telecommunications Engineering — Section 071
Test Automation — Section 255
Threat Intelligence — Section 207
Threat Modeling — Section 208
Tooling Engineering — Section 274
Tracing — Section 316
Training — Section 338
Trust and Safety — Section 217
UI Design — Section 083
Unit Testing — Section 233
Usability Testing — Section 249
Usage Metering — Section 365
User Research — Section 015
User Validation — Section 106

UX Architecture — Section 081
UX Design — Section 082
UX Research — Section 080
UX Strategy — Section 079
Validation — Section 257
Vendor Management — Section 349
Verification — Section 256
Version Management — Section 388
Video Engineering — Section 143
Virtualization Engineering — Section 281
Visual Design — Section 084
Voice Interface Design — Section 088
VR Design — Section 092
Vulnerability Management — Section 209
Web Engineering — Section 121
XR Design — Section 093

CONCEPT CROSS-INDEX — CURRENT DRAFTS

Affected parties — Sections 001, 002, 004, 006, 014, 015, 016, 018
AI capability versus authority — Sections 001–026; especially 004, 006, 010, 011, 014–020
Anti-metrics — Sections 002, 004, 005, 006, 007, 010
Assumptions and unknowns — Sections 001–026
Bounded contexts — Section 016
Business rules — Sections 011, 016
Capability models — Sections 005, 006, 007, 011, 016
Commitments — Sections 005, 007, 008, 009, 010, 014
Conditional feasibility — Section 017
Consent and withdrawal — Sections 012, 014, 015
Control effectiveness — Section 018
Critical journeys — Sections 006, 008, 010, 011, 014, 015, 016
Disqualifying conditions — Sections 017, 018
Dissent — Sections 001–026
Domain events and state transitions — Section 016
Evidence-before-claims — Sections 001–026
Experiment boundaries — Sections 003, 006, 012, 015
Feasibility dimensions — Section 017
Feasibility experiments and pilots — Section 017
Full lifecycle cost and supportability — Section 017
Human final authority — Sections 001–026
Human-AI relationship boundaries — Sections 002, 004, 005, 006, 010, 014, 015, 016, 020
Inherent and residual risk — Section 018
Market evidence — Sections 005, 006, 010, 012, 013
Mission invariants — Sections 004–010
Owner cockpit and user control — Sections 001–026
Participant protection — Sections 012, 014, 015
Portfolio dependency and concentration — Section 007
Product capability state — Sections 006, 010
Program integration and benefits — Section 008
Project evidence and closure — Section 009
Recovery and reconsideration — Sections 001–026
Risk acceptance and expiry — Section 018
Risk correlation and cascade — Section 018
Risk treatment and safe halt — Section 018
Source identity and freshness — Sections 001–026
Stakeholder authority and commitments — Section 014
Synthetic versus human evidence — Sections 012, 015

Terminology and semantic boundaries — Sections 011, 016
Treatment-created and transferred risk — Section 018
User jobs and mental models — Section 015

Acceptance criteria and evidence methods — Section 019
AI-proposed versus approved requirements — Section 019
Bidirectional requirements traceability — Section 019
Mandatory constraints versus priority — Sections 017–020
Requirement applicability and variants — Section 019
Requirement baseline and invalidation — Section 019
Requirement criticality versus priority — Section 019
Requirement quality and atomicity — Section 019
Requirements validation and verification — Section 019
Source-to-requirement derivation — Section 019

System baseline and configuration identity — Section 020
System boundaries and context — Section 020
System elements and configuration items — Section 020
Requirements allocation — Sections 019–020
Operational concept and mission threads — Section 020
Interface control and compatibility — Section 020
Measures of effectiveness and performance — Section 020
Systems integration strategy — Section 020
System verification versus validation — Section 020
Enabling systems and lifecycle support — Section 020
Human-AI teaming and autonomy boundaries — Section 020
Whole-system failure and recovery — Section 020
Technical baselines and change invalidation — Section 020
Architecture decision records — Sections 021, 022
Architecture principles and standards — Section 021
Architecture roadmap and transition states — Section 021
Capability-to-investment traceability — Section 021
Current, target, and transition architecture — Sections 021, 022
Enterprise operating model — Section 021
Governed action boundary — Section 022
Requirements-to-architecture traceability — Sections 019–022
Solution components and interfaces — Section 022
Value streams — Section 021

Architecture invariants — Section 023
Architecture-to-evidence traceability — Section 023
Failure domains and blast radius — Section 023
State ownership and source of truth — Sections 022–024
Systems architecture elements and interfaces — Section 023
Systems architecture trust boundaries — Section 023
Whole-system degraded modes and recovery — Sections 020, 022, 023
Anti-corruption layers — Section 024
Bounded-context architecture — Sections 016, 024
Context maps and relationships — Section 024
Domain authority and decision rights — Sections 016, 024
Domain model ownership — Section 024
Domain translation and published language — Section 024
Semantic contracts — Section 024
Temporal, jurisdictional, and effective-date semantics — Sections 016, 024
Business architecture current, target, and transition states — Section 025
Business architecture decision rights and accountability — Section 025
Business architecture operating model — Sections 021, 025
Business information concepts and evidence ownership — Section 025

Business value propositions and outcomes — Section 025
Business value streams and capability alignment — Sections 021, 025, 026
Capability architecture and taxonomy — Section 026
Capability catalog and ownership — Section 026
Capability current-state assessment — Section 026
Capability definitions independent of realization — Section 026
Capability dependencies and enabling capabilities — Section 026
Capability gaps, levels, and lifecycle states — Section 026
Capability increments and transition roadmap — Section 026
Capability realization across people, process, information, and technology — Section 026
Capability-to-initiative and investment traceability — Sections 021, 025, 026
Capability versus feature, system, project, or model — Section 026
Capability versus authority and permission — Sections 020, 025, 026
Operating model and organizational realization — Section 025
Value proposition evidence and anti-metrics — Section 025



CROSS REFERENCE REGISTER

CODEX/90_REFERENCE/003_CROSS_REFERENCE_REGISTER.txt

SHA-256 8399608d4b84d3a901f080cbd2b95b15e6845b07d970202d9206d682c6e6fd39

17,645 BYTES / PRESERVED SOURCE IDENTITY

CROSS-REFERENCE REGISTER — WORKING

This register is expanded as sections are authored. Section files remain the controlling source for detailed dependencies, evidence, failure, recovery, and completion requirements.

SECTION 001 — RESEARCH

Upstream: human objective, authority, initial scope, supplied source state.
Downstream: 002–022 and all later architecture, design, engineering, security, validation, operation, maintenance, and retirement sections as applicable.
Reopen triggers: contradictory evidence, failed feasibility, user findings, incidents, changed source state, new law or constraint, post-implementation evidence.

SECTION 002 — FORESIGHT / FUTURES

Upstream: 001 Research, human decision question, authority, current baseline, declared horizons, supplied source state.
Downstream: 003–022, architecture, risk, safety, governance, roadmap, maintenance, modernization, deprecation, continuity, and retirement sections as applicable.
Reopen triggers: changed drivers or signals, failed assumptions, discontinuity, changed authority or jurisdiction, major incidents, capability shifts, changed dependencies, downstream contradiction.

SECTION 003 — INNOVATION

Upstream: 001 Research; 002 Foresight / Futures where horizon matters; authorized opportunity and experiment boundary.
Downstream: 004–022, product, architecture, experiment, IP, security, safety, validation, and lifecycle decisions.
Reopen triggers: failed hypothesis, prior-art conflict, participant harm, invalid experiment, dependency change, new evidence, mission or risk conflict.

SECTION 004 — MISSION STRATEGY

Upstream: 001 Research, founding or owner intent, affected-party evidence, legitimate mission authority.
Downstream: 005–010, requirements, governance, architecture, product, AI safety, policy, ethics, operations, and retirement.
Reopen triggers: mission drift, changed legitimacy, harmful metric effects, unresolved value conflict, new affected parties, changed evidence, authorized amendment.

SECTION 005 — BUSINESS STRATEGY

Upstream: 004 Mission Strategy, 001–003 evidence, market and stakeholder context, authorized strategy owner.
Downstream: 006–010, portfolio, programs, products, capabilities, architecture, finance, operations, procurement, and lifecycle.
Reopen triggers: failed economics, changed demand, cost, capacity, law, provider, partner, mission, incentive, or resource evidence.

SECTION 006 — PRODUCT STRATEGY

Upstream: 004 Mission Strategy, 005 Business Strategy, research, market, stakeholder, user, domain, feasibility, and risk evidence.
Downstream: 007–022, product management, HCI, UX, architecture, engineering, AI, release, feedback, deprecation, and retirement.
Reopen triggers: invalid user need, capability drift, provider change, failed validation, incident, accessibility or trust failure, changed mission or market.

SECTION 007 — PORTFOLIO MANAGEMENT

Upstream: mission, business and product strategies; current initiative, capacity, risk, dependency, and obligation evidence.

Downstream: 008–010, programs, projects, products, enterprise and platform architecture, capacity, roadmap, migration, and retirement.
Reopen triggers: changed capacity, shared dependency, correlated incident, failed benefit, new obligation, portfolio imbalance, strategy change.

SECTION 008 — PROGRAM MANAGEMENT

Upstream: authorized portfolio or strategic outcome, participating workstreams, shared dependencies, resources, and governance.
Downstream: 009–022, integration, configuration, release, change, operations, adoption, benefits, and post-implementation review.
Reopen triggers: workstream failure, version mismatch, unresolved dependency, resource change, integration failure, transition failure, benefit shortfall.

SECTION 009 — PROJECT MANAGEMENT

Upstream: approved project objective, requirements or defined outcome, sponsor, resources, dependencies, constraints, and acceptance.
Downstream: implementation, design, engineering, testing, release, operational transition, maintenance, and review sections.
Reopen triggers: scope change, failed requirement, unavailable resource, blocked dependency, failed test, incident, rejected acceptance, changed target state.

SECTION 010 — PRODUCT MANAGEMENT

Upstream: 006 Product Strategy, current product and capability state, user and market evidence, operational and lifecycle evidence.
Downstream: 011–022, roadmap, requirements, design, delivery, release, feedback, analytics, support, migration, deprecation, and retirement.
Reopen triggers: changed user evidence, capability or provider drift, failed outcome, incident, support pattern, regulatory change, lifecycle obligation.

SECTION 011 — BUSINESS ANALYSIS

Upstream: strategy, product, project or policy question; stakeholder, user, organizational, process, data, and current-state evidence.
Downstream: 016 Domain Analysis, 017 Feasibility Analysis, 018 Risk Management, 019 Requirements Engineering, business, capability, process, information, and data architecture.
Reopen triggers: changed business rule, policy, authority, current state, stakeholder validation, terminology, process, data, or target outcome.

SECTION 012 — MARKET RESEARCH

Upstream: decision question, population and category definitions, lawful method, source and participant protections.
Downstream: business and product strategy, product management, portfolio, feasibility, pricing, positioning, experimentation, and roadmap decisions.
Reopen triggers: expired source, changed market definition, sampling defect, new population, regulatory or economic change, contradictory evidence.

SECTION 013 — COMPETITIVE INTELLIGENCE

Upstream: authorized intelligence question, lawful collection boundary, mission and product criteria, current internal capability evidence.
Downstream: strategy, product, procurement, security, partnerships, positioning, roadmap, and risk decisions.
Reopen triggers: source expiration, product or tier change, independent test, incident, market entry, provider shift, comparison defect, legal constraint.

SECTION 014 — STAKEHOLDER MANAGEMENT

Upstream: objective, scope, affected-system boundary, identity and authority model, privacy, consent, accessibility, and legal constraints.

Downstream: user research, business analysis, requirements, product, program, project, governance, communication, change, incident, and lifecycle work.
Reopen triggers: new affected party, changed impact, consent withdrawal, grievance, broken commitment, incident, change, migration, deprecation, retirement.

SECTION 015 — USER RESEARCH

Upstream: 001 Research, product or system decision, stakeholder map, participant protections, current journey and product state.
Downstream: 016–022, HCI, UX strategy and research, interaction, accessibility, product, AI, trust and safety, usability testing, and validation.
Reopen triggers: changed user population, product or prototype version, accessibility context, invalid sample or consent, contradictory behavior, incident, failed validation.

SECTION 016 — DOMAIN ANALYSIS

Upstream: 001 Research, 011 Business Analysis, 014 Stakeholder Management, 015 User Research, authoritative domain sources and experts.
Downstream: 017–060, Requirements Engineering, Systems Engineering, Domain and Business Architecture, Data Modeling, API, events, policy, AI, testing, operations, and lifecycle.
Reopen triggers: changed definition, rule, invariant, authority, jurisdiction, effective date, state model, boundary, source of truth, implementation contradiction, or domain incident.

SECTION 017 — FEASIBILITY ANALYSIS

Upstream: 001–016, authorized objective and options, current-state evidence, constraints, requirements, dependencies, resources, and applicable specialist evidence.
Downstream: 018–060, Requirements Engineering, Systems Engineering, architecture, engineering, security, privacy, safety, accessibility, legal, testing, operations, procurement, maintenance, migration, and retirement.
Reopen triggers: changed objective, requirement, environment, scale, workload, data, model, provider, resource, budget, schedule, law, user population, dependency, control, incident, failed experiment, or new evidence.

SECTION 018 — RISK MANAGEMENT

Upstream: 001–017, objective and affected-party context, feasibility conditions, architecture and dependency evidence, incidents, controls, constraints, and risk authority.
Downstream: 019–060 and all engineering, security, privacy, safety, accessibility, testing, delivery, operations, maintenance, decommissioning, and retirement sections through risk treatments, thresholds, controls, monitoring, and acceptance conditions.
Reopen triggers: changed exposure, affected party, objective, environment, architecture, dependency, data, model, provider, law, control effectiveness, incident, near miss, telemetry, acceptance expiry, treatment failure, or new evidence.

SECTION 019 — REQUIREMENTS ENGINEERING

Upstream: Sections 001–018, authenticated objective and authority, source and current-state evidence, affected-party and user findings, domain meaning, feasibility conditions, risk treatments, mandatory constraints, acceptance boundaries, and human decision points.
Downstream: Section 020 and all enterprise, solution, systems, domain, business, operating-model, process, capability, data, information, application, technology, software, platform, infrastructure, integration, API, event, runtime, security, identity, privacy, governance, compliance, safety, reliability, resilience, accessibility, content, analytics, AI, machine-learning, robotics, embedded, hardware, design, engineering, testing, verification, validation, delivery, operation, support, maintenance, migration, deprecation, lifecycle-governance, and retirement work as applicable.
Reopen triggers: changed objective, authority, stakeholder, affected party, user evidence, domain rule, law, policy, contract, environment, system boundary, feasibility condition, risk treatment, architecture, interface, data, model, provider, implementation deviation, failed test, incident, acceptance evidence, lifecycle obligation, or source version.

SECTION 020 — SYSTEMS ENGINEERING

Upstream: Sections 001–019, exact approved requirements baseline, system objective and boundary, affected-party and user evidence, domain model, feasibility conditions, risk treatments, mandatory constraints, acceptance criteria, verification methods, authority, and current-state evidence.
Downstream: Sections 021–060 and all specialist architecture, design, engineering, acquisition, integration, testing, verification, validation, transition, deployment, operations, support, maintenance, migration, decommissioning, lifecycle-governance, and retirement work through system context, allocations, elements, interfaces, measures, baselines, evidence, and lifecycle obligations.
Reopen triggers: changed objective, owner, authority, requirement, system boundary, affected party, operational concept, environment, lifecycle stage, architecture, function, element, interface, data, model, provider, supplier, dependency, configuration, measure, feasibility condition, risk treatment, law, policy, contract, implementation deviation, integration finding, failed verification, failed validation, incident, operational evidence, maintenance finding, migration effect, or retirement constraint.

SECTION 021 — ENTERPRISE ARCHITECTURE

Upstream: Sections 001–020, enterprise mission and operating model, portfolio and product strategies, approved requirements and systems baselines, current-state evidence, affected-party obligations, capability and value-stream evidence, feasibility conditions, risk treatments, authority, and lifecycle constraints.
Downstream: Section 022 and all business, capability, process, functional, operational, information, data, application, software, platform, infrastructure, cloud, integration, security, governance, reliability, resilience, operations, modernization, migration, deprecation, lifecycle-governance, and retirement sections through enterprise principles, standards, shared capabilities, target states, transition architectures, roadmaps, exceptions, and decision rights.
Reopen triggers: changed mission, strategy, enterprise boundary, operating model, capability, value stream, investment, organization, requirement, system allocation, shared service, standard, data authority, identity, security, supplier, law, risk, incident, migration evidence, exception, target-state assumption, operational result, or retirement obligation.

SECTION 022 — SOLUTION ARCHITECTURE

Upstream: Sections 001–021, approved objective and requirements baseline, system context and allocations, enterprise principles and standards, domain rules, current-state code and runtime evidence, risks, feasibility conditions, environments, providers, user and owner journeys, acceptance criteria, and lifecycle obligations.
Downstream: Sections 023–060 and all specialist architecture, design, engineering, testing, verification, validation, deployment, runtime, operations, support, maintenance, migration, deprecation, lifecycle-governance, and retirement work through components, interfaces, contracts, flows, data and state ownership, trust boundaries, deployment, recovery, observability, evidence, and implementation sequencing.
Reopen triggers: changed objective, requirement, authority, enterprise constraint, system allocation, domain rule, component, interface, schema, data source, provider, environment, workload, performance, security, privacy, accessibility, resilience, deployment, migration, test, verification, validation, incident, operational evidence, support burden, maintenance constraint, or retirement decision.

SECTION 023 — SYSTEMS ARCHITECTURE

Upstream: 001-022, especially approved requirements, system context, operational concept, enterprise principles and standards, solution decisions, domain analysis, feasibility conditions, risks, and current-state evidence.
Downstream: 024-060 specialist architecture; 107-230 engineering; 231-257 testing, verification, and validation; 258-300 delivery and infrastructure; 301-332 runtime and operations; 374-406 maintenance through retirement.
Reopen triggers: changed requirement, domain meaning, system boundary, element responsibility, interface, state owner, authority, trust boundary, dependency, environment, failure evidence, architecture decision, provider, incident, migration, or lifecycle obligation.

SECTION 024 — DOMAIN ARCHITECTURE

Upstream: 001-023, especially 011 Business Analysis, 014 Stakeholder Management, 015 User Research, 016 Domain Analysis, 019 Requirements Engineering, 020 Systems Engineering, and 021-023 architecture outputs.
Downstream: 025-033 business, capability, process, functional, operational, information, data, application, and software architecture; 041-043 integration, API, and event architecture; 048-049 governance and compliance architecture; 054-057 content, knowledge, AI, and machine-learning architecture; 107-230 engineering; testing, operations, and lifecycle sections.

Reopen triggers: changed definition, context boundary, source hierarchy, rule, invariant, state, event, authority, jurisdiction, effective date, translation, interface, data ownership, policy, domain incident, migration evidence, or downstream semantic contradiction.

SECTION 025 — BUSINESS ARCHITECTURE

Upstream: Sections 001–024, especially mission, business and product strategy, portfolio and program context, stakeholder and user evidence, domain meaning, feasibility, risk, requirements, Systems Engineering, and enterprise, solution, systems, and domain architecture.
Downstream: Section 026 and all process, functional, operational, information, organization, product, service, application, technology, platform, integration, security, privacy, safety, accessibility, AI, delivery, change, adoption, measurement, operations, support, maintenance, modernization, deprecation, and retirement work through value streams, capabilities, decision rights, operating model, measures, and evidence.
Reopen triggers: changed mission, strategy, stakeholder, affected party, user, market, domain, requirement, value proposition, value stream, capability, decision right, policy, organization, information, product, service, partner, supplier, resource, cost, capacity, law, risk, system, incident, measure, adoption, evidence, or lifecycle state.

SECTION 026 — CAPABILITY ARCHITECTURE

Upstream: Sections 001–025, especially mission and business strategy, portfolio and product context, Business Analysis, Domain Analysis, Feasibility Analysis, Risk Management, Requirements Engineering, Systems Engineering, Enterprise Architecture, Systems Architecture, Domain Architecture, and Business Architecture.
Downstream: Sections 027–060 and organization, workforce, process, functional, operational, information, data, application, technology, platform, infrastructure, integration, security, privacy, safety, accessibility, AI, portfolio, investment, testing, operations, support, maintenance, modernization, deprecation, and retirement work through capability definitions, realization, dependencies, levels, gaps, measures, increments, evidence, and lifecycle states.
Reopen triggers: changed mission, strategy, outcome, value stream, requirement, capability definition, owner, authority, realization, demand, capacity, performance, quality, dependency, workforce, supplier, system, model, data, environment, law, policy, risk, control, incident, evidence, investment, benefit, or lifecycle state.



SOURCE REGISTER

CODEX/90_REFERENCE/004_SOURCE_REGISTER.txt

SHA-256 56113af5c3233d6d3f1dbbf6d3bd4b9425c739134db689333d07922fea93ef05

1,069 BYTES / PRESERVED SOURCE IDENTITY

SOURCE REGISTER

These files are imported source inputs. Their inclusion does not make every source statement a finalized Codex requirement; applicable content is integrated and reconciled through authored Codex files.

Pasted text(20).txt
SHA-256: dc324a610c65ea7df9eff2af82a6dd69e6ea997982422efc7dfe56005264c2d4

Thirsty’s Standard v3.pdf
SHA-256: 798d9bd614c16ba25d632b5292ea732d756aeb65e12ecab97c66a784df376e5b

Thirstys_Agents_101_v1.3(3).pdf
SHA-256: 3dd8584f68c08613b93f3ef57fcd42bbb50c1c9b7d4cb629d092d326cff6910f

Thirstys_Owners_Cockpit_Standard_v1.0_Command_Edition(2).pdf
SHA-256: e8484ea46ce017fda427bc748627e7313f536c4a6a41248832b6b069238f8b0b

Thirstys_Standard_V4_Design_Blueprint_v0.1.0-draft.pdf
SHA-256: b691064c4bfb46745d6a5ccad2f7137209a0d678f48d9852a7987e1011c26aa5

Thirstys_Standard_V4_Canonical_Source_Stack_v0.2.0-draft.zip
SHA-256: 58e882647bde4365e35f178bbab7680d22f348e4eec8bf1396c564cdd0b7353d

Thirstys_Standard_V4_Canonical_Source_Stack_v0.2.0-draft.zip.sha256
SHA-256: 77b410ce70f57f0fe318b53d5bef0d8ff1db2ff0091567fb0574bc176db9add2



INTEGRATION CHECKPOINT 001 018

CODEX/90_REFERENCE/005_INTEGRATION_CHECKPOINT_001_018.txt

SHA-256 81f8d36dfdcc4d1c15303bbda09d75ea4cabc8e03cb17bf47de9f0c447e609d3

5,776 BYTES / PRESERVED SOURCE IDENTITY

THIRSTY'S CODEX — INTEGRATION CHECKPOINT 001–018

STATUS: COMPLETED HISTORICAL WORKING-DRAFT CHECKPOINT

CURRENT-STATE NOTICE

This file preserves the status and evidence available when the 001–018 checkpoint was created. The current corpus state is maintained in MANIFEST.txt and CONTINUITY_MAP.txt. All 406 lifecycle sections are now CONTENT-COMPLETE for the authoring scope.

SCOPE

Sections 001 through 018, covering Research through Risk Management, plus the shared governance files, front matter, glossary, index, cross-reference register, source register, manifest, continuity map, role corpus, and expected-output contracts.

PURPOSE

Confirm that the first lifecycle block forms a coherent upstream foundation for Requirements Engineering without hiding status, rewriting existing roles, collapsing human and AI authority, or allowing feasibility and risk obligations to disappear before architecture and implementation.

STRUCTURAL RESULTS

- All 406 lifecycle section files are present.
- Sections 001–018 are marked DRAFTED and contain substantive content.
- At the time this checkpoint was recorded, Sections 019–406 remained marked NOT YET DRAFTED. The current whole-Codex status is maintained in MANIFEST.txt and CONTINUITY_MAP.txt.
- Every drafted section contains all twenty-four required canonical section headings.
- The twenty source-defined role files and twenty expected-output contracts remain present.
- Front matter, shared governance, reference, source, manifest, continuity, and integrity files remain connected to the canonical tree.

CHAIN INTEGRATION

The first block now establishes the following governed flow:

Research establishes evidence and uncertainty.
Foresight / Futures tests horizons and plausible change.
Innovation bounds experiments and novelty claims.
Mission Strategy defines legitimate purpose and invariants.
Business Strategy defines participation, value, capabilities, and economic direction.
Product Strategy defines intended users, outcomes, product boundaries, and lifecycle intent.
Portfolio Management governs competing initiatives and shared capacity.
Program Management coordinates outcomes that span workstreams.
Project Management governs bounded delivery and closure.
Product Management governs continuing product decisions and learning.
Business Analysis translates objective and current state into decision-relevant needs.
Market Research examines external populations and conditions.
Competitive Intelligence examines lawful comparative evidence.
Stakeholder Management establishes affected parties, authority, commitments, and engagement.
User Research grounds human jobs, behavior, accessibility, trust, failure, and recovery.
Domain Analysis establishes canonical meaning, rules, states, authority, and boundaries.
Feasibility Analysis determines whether options can be realized under actual conditions.
Risk Management turns uncertainty and consequence into governed treatment, controls, monitoring, acceptance, and escalation.

CROSS-CUTTING DETERMINATIONS

- Human authority is retained for objectives, values, commitments, consent, resource commitments, risk acceptance, and final consequential decisions.
- AI systems may analyze, propose, test, explain, monitor, and assist within bounded authority; capability does not confer permission or legitimacy.
- Facts, observations, measurements, estimates, assumptions, hypotheses, scenarios, preferences, unknowns, recommendations, decisions, actions, outcomes, and evidence remain distinguishable.
- Source identity, version, environment, scope, time, and evidence boundaries are required for material claims.
- Dissent and contradictory evidence remain visible.
- Failure, denial, degradation, recovery, rollback, revocation, and retirement are treated as lifecycle paths rather than exceptional afterthoughts.
- Owner-facing control requires understandable state, limitations, evidence, consequence, recovery, and stop paths.
- Feasibility conditions and risk treatments are explicitly handed to Requirements Engineering rather than left as narrative advice.

AUTOMATED CHECKS REQUIRED FOR RELEASE

- Exact count of 406 section files.
- Exact drafted count of 18 and placeholder count of 388.
- Presence of every canonical heading in Sections 001–018.
- At checkpoint creation, absence of NOT YET DRAFTED markers in Sections 001–018.
- At checkpoint creation, presence of NOT YET DRAFTED markers in Sections 019–406; this historical condition is not the current corpus status.
- Exact count of twenty preserved source-role files and twenty imported expected-output contracts, with any Codex-native role extensions tracked separately.
- ZIP integrity and safe member paths.
- Internal SHA-256 manifest verification after clean extraction.
- Matching outer SHA-256 sidecar.

OPEN INTEGRATION LIMITS

- DRAFTED is not SOURCE-RECONCILED, HOSTILE-REVIEWED, OWNER-ACCEPTED, IMPLEMENTED, TESTED, or VERIFIED.
- General methods in the first block require later reconciliation with specialist discipline sections and qualified current sources.
- At checkpoint creation, later lifecycle sections had not yet been authored. They are now CONTENT-COMPLETE, but the 001–018 block has not yet completed final whole-Codex source reconciliation or hostile review against them.
- The V4 source remains draft and unratified.

CHECKPOINT DECISION

The first block is structurally and conceptually ready to hand forward to Section 019 — Requirements Engineering as a working draft foundation. Advancement remains bounded by the limitations above.

NEXT SAFE ACTION

Preserve this checkpoint as historical evidence. Use the current whole-Codex roadmap to conduct checkpoint integration, source reconciliation, hostile review, and evidence-backed reopening.

REMAINING SECTION INTEGRATION ROADMAP

CODEX/90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt

SHA-256 3e1689503cc06f08585d30def9704a8f346a95bd6cb7260f38d80f697d43504e

11,459 BYTES / PRESERVED SOURCE IDENTITY

THIRSTY'S CODEX — WHOLE-CODEX INTEGRATION AND ASSURANCE ROADMAP

PURPOSE

Provide a durable, repository-local plan for finishing the integration and assurance of the content-complete 406-section corpus without relying on chat history, undocumented memory, or an agent's private context.

This roadmap governs checkpoint integration, source reconciliation, hostile review, reopening, support-file maintenance, verification, packaging, owner-acceptance boundaries, and final whole-Codex release preparation. Section authoring is complete; the remaining work is assurance.

CURRENT CANONICAL BASELINE

- All 406 lifecycle section files exist.
- Sections 001–406 are marked STATUS: CONTENT-COMPLETE.
- No section remains DRAFTED or NOT YET DRAFTED.
- Every section contains the canonical section headings and substantive content for the declared authoring scope.
- The historical checkpoint for Sections 001–018 exists as a working-draft integration record.
- Formal checkpoints 019–060 through 402–406 and the final whole-Codex checkpoint remain open.
- Twenty source-defined role files and twenty imported expected-output contracts remain preserved.
- One separately identified Codex-native Guidebook Architect role and matching expected-output contract are present.

STATUS BOUNDARY

CONTENT-COMPLETE means the authoring pass is complete for the declared section scope. It does not mean SOURCE-RECONCILED, HOSTILE-REVIEWED, OWNER-ACCEPTED, IMPLEMENTED, TESTED, VERIFIED, READY, RATIFIED, or PUBLISHED.

A checkpoint may promote a section only when the exact gate criteria and evidence are recorded. Owner acceptance may be assigned only by the accountable human owner over exact content and artifact identity.

ASSURANCE PRINCIPLES

- Preserve the human objective, terminology, values, authority, and final decision rights.
- Preserve source identity, provenance, version, scope, time, method, and evidence boundaries.
- Keep fact, source statement, inference, assumption, recommendation, decision, implementation, test, verification, and acceptance distinct.
- Unknown means unknown. Missing evidence must not be repaired with confident prose.
- Preserve dissent, contradictory evidence, exceptions, and unresolved gaps.
- Reopen earlier sections when later evidence reveals a material defect, contradiction, omission, or stale assumption.
- Use the smallest sufficient role set and keep execution, challenge, verification, and approval independent where consequence requires it.
- A denial or block must name the exact blocker, safe work that may continue, the smallest resolution path, authority required, and evidence needed for reconsideration.

FORMAL CHECKPOINT SEQUENCE

1. Existing historical record: 001–018 — Research through Risk Management.
File: 90_REFERENCE/005_INTEGRATION_CHECKPOINT_001_018.txt
Action: preserve as historical evidence; reopen affected sections during later checkpoints when required.

2. Checkpoint 019–060 — Requirements Engineering through Hardware Architecture.
Create: 90_REFERENCE/007_INTEGRATION_CHECKPOINT_019_060.txt

3. Checkpoint 061–106 — Electrical Engineering through Design Validation.
Create: 90_REFERENCE/008_INTEGRATION_CHECKPOINT_061_106.txt

4. Checkpoint 107–150 — Programming Language Design through Smart Contracts Engineering.
Create: 90_REFERENCE/009_INTEGRATION_CHECKPOINT_107_150.txt

5. Checkpoint 151–188 — Database Engineering through Human-in-the-Loop Operations.
Create: 90_REFERENCE/010_INTEGRATION_CHECKPOINT_151_188.txt

6. Checkpoint 189–218 — Cybersecurity Engineering through Privacy Engineering.
Create: 90_REFERENCE/011_INTEGRATION_CHECKPOINT_189_218.txt

7. Checkpoint 219–257 — Legal through Test Automation.
Create: 90_REFERENCE/012_INTEGRATION_CHECKPOINT_219_257.txt

8. Checkpoint 258–300 — Configuration Management through Production Promotion.
Create: 90_REFERENCE/013_INTEGRATION_CHECKPOINT_258_300.txt

9. Checkpoint 301–332 — Site Reliability Engineering through Command and Control.
Create: 90_REFERENCE/014_INTEGRATION_CHECKPOINT_301_332.txt

10. Checkpoint 333–373 — Technical Support through Roadmap Management.
Create: 90_REFERENCE/015_INTEGRATION_CHECKPOINT_333_373.txt

11. Checkpoint 374–401 — Maintenance Strategy through Environmental Disposal or Recycling.
Create: 90_REFERENCE/016_INTEGRATION_CHECKPOINT_374_401.txt

12. Checkpoint 402–406 — Post-Implementation Review through System Retirement.
Create: 90_REFERENCE/017_INTEGRATION_CHECKPOINT_402_406.txt

13. Final checkpoint 001–406 — Whole-Codex integration, assurance, and release boundary.
Create: 90_REFERENCE/018_WHOLE_CODEX_INTEGRATION_CHECKPOINT_001_406.txt

CHECKPOINT WORKFLOW

For each checkpoint:

1. Establish exact baseline identity.
- Record archive or commit identity, internal manifest result, working directory, and date.
- Confirm section count, status count, role count, contract count, and safe archive paths.
- Preserve pre-change hashes for source-defined roles, imported contracts, and source artifacts.

2. Read the complete checkpoint range and its dependencies.
- Read upstream sections named by prerequisites and related-section references.
- Read downstream handoff targets where meaning, allocation, evidence, or authority crosses the checkpoint boundary.
- Read applicable shared governance, role contracts, source register, glossary, index, cross-reference register, manifest, continuity map, and prior checkpoint records.

3. Reconcile structure and meaning.
- Check terminology, definitions, duplicated ownership, missing ownership, conflicting requirements, lifecycle state, authority, evidence, failure, recovery, and handoff consistency.
- Check that requirements flow into architecture, implementation, testing, operations, maintenance, and retirement without disappearing.
- Check that human authority, AI boundaries, informed consent, capability-versus-permission, reversibility, dignity, non-deception, traceable delegation, and owner operability remain coherent.

4. Reconcile sources.
- Identify supplied sources controlling each material claim.
- Record source version, authority, scope, currency, conflicts, and unresolved gaps.
- Distinguish source-derived requirements from Codex synthesis, recommendation, example, and open question.
- Use current external sources only when authorized and record them in the source register.

5. Conduct hostile review.

- Challenge omissions, contradictions, false completeness, vague applicability, untestable criteria, hidden authority, unsupported universal claims, security and privacy gaps, inaccessible user paths, missing failure handling, weak recovery, stale evidence, and circular verification.
- Preserve each material finding with severity, confidence, evidence, affected sections, owner, disposition, and reopening condition.

6. Reopen and correct only what evidence requires.
- Record exact sections reopened and why.
- Preserve prior state and correction history.
- Return a section to CONTENT-COMPLETE after the material revision is fully authored.
- Promote to SOURCE-RECONCILED or HOSTILE-REVIEWED only when the complete gate is actually satisfied and evidenced.

7. Update canonical support files.
- MANIFEST.txt
- CONTINUITY_MAP.txt
- README.txt
- 90_REFERENCE/001_GLOSSARY.txt
- 90_REFERENCE/002_INDEX.txt
- 90_REFERENCE/003_CROSS_REFERENCE_REGISTER.txt
- 90_REFERENCE/004_SOURCE_REGISTER.txt
- this roadmap progress ledger
- applicable checkpoint file
- front-matter glossary, index, and table of contents when affected

8. Verify.
- Confirm 406 uniquely numbered section files.
- Confirm every section filename and title header agree.
- Confirm every section retains the canonical headings and no authoring placeholder.
- Confirm exact status counts.
- Confirm source-defined roles and imported contracts remain unchanged unless the owner explicitly authorizes a source replacement.
- Confirm internal SHA-256 records in the working tree and a clean extraction.
- Run ZIP integrity and safe-path checks.
- Generate and verify the outer SHA-256 sidecar against the exact ZIP delivered.

CHECKPOINT OUTPUT CONTRACT

Every checkpoint file must contain:

- checkpoint identity, range, owner, date, and baseline artifact identity;
- included and excluded scope;
- sources examined and source gaps;
- terminology and definition findings;
- authority, human-AI, evidence, failure, recovery, and handoff findings;
- contradictions, duplicates, omissions, stale references, and affected sections;
- hostile-review findings and dissent ledger;
- reopened sections and corrections;
- status transitions with evidence;
- verification commands and results;
- unresolved conditions and decision owners;
- next safe action and safe-to-continue state.

REOPENING RULE

Reopening is normal assurance work. A section must be reopened when material evidence shows that its content is incomplete, contradictory, incorrectly scoped, unsupported, unusable, unsafe, inaccessible, or inconsistent with governing source or later lifecycle obligations.

A reopening record must identify:

- section and exact affected claim;
- trigger and evidence;
- consequence if unchanged;
- authority for the correction;
- related sections and artifacts;
- required revision and acceptance method;
- prior and new status;
- evidence required to close the reopening.

FINAL WHOLE-CODEX CHECKPOINT

After checkpoint 402–406, perform the final 001–406 integration:

- reconcile global terminology, definitions, lifecycle states, role names, authority, evidence, consent, security, privacy, safety, accessibility, reliability, resilience, operations, recovery, maintenance, deprecation, disposal, and retirement;
- check every upstream-to-downstream handoff and every reopen trigger;
- produce a global source-reconciliation register, hostile-review record, dissent ledger, unresolved-gap register, and owner-decision register;
- verify navigation, table of contents, glossary, index, cross-references, source register, status counts, hashes, and package identity;
- distinguish content completion, source reconciliation, hostile review, independent verification, owner acceptance, ratification, and publication;
- require explicit owner acceptance over the exact final archive hash before assigning OWNER-ACCEPTED.

PROGRESS LEDGER

- Section authoring 001–406: CONTENT-COMPLETE.
- Placeholders: 0.
- Historical checkpoint 001–018: completed as a working-draft checkpoint; later assurance may reopen it.
- Checkpoint 019–060: pending.
- Checkpoint 061–106: pending.
- Checkpoint 107–150: pending.
- Checkpoint 151–188: pending.
- Checkpoint 189–218: pending.
- Checkpoint 219–257: pending.
- Checkpoint 258–300: pending.
- Checkpoint 301–332: pending.
- Checkpoint 333–373: pending.
- Checkpoint 374–401: pending.
- Checkpoint 402–406: pending.
- Final checkpoint 001–406: pending.
- SOURCE-RECONCILED sections: 0.
- HOSTILE-REVIEWED sections: 0.
- OWNER-ACCEPTED sections: 0.

NEXT SAFE ACTION

Begin Checkpoint 019–060. Use Sections 001–018 as upstream source state, read the full 019–060 range, reconcile sources and cross-section obligations, conduct hostile review, reopen only evidence-backed defects, create 90_REFERENCE/007_INTEGRATION_CHECKPOINT_019_060.txt, update canonical support files, and rebuild hashes and the archive only after the checkpoint record is stable.

SAFE TO CONTINUE: YES

SOURCE WITNESS 503

Agi Human Working Agreement

SHA-256 f74b21fb29abc94ba5aedb88cdc553b4aa3262f9159e4d40cefa9ef46175fb3a

AGI-Human Working Agreement

Status: PROPOSED - operational aid, not a ratified Codex requirement
Applies to: Work performed by a human and one or more AI agents on Thirsty's Codex
Owner: The accountable human owner of Thirsty's Codex

Purpose

This agreement makes the collaboration itself governable. It gives the human and the AI a shared operating language for intent, authority, evidence, uncertainty, disagreement, execution, recovery, and completion.

It supplements the canonical requirements in `01_SHARED_GOVERNANCE/` and does not rewrite or extend the twenty source-defined agent roles.

Shared Objective

The shared objective is to complete Thirsty's Codex as a truthful, coherent, source-reconciled, independently challenged, owner-accepted lifecycle framework for responsible human-AI work.

The project is not complete because all files exist, all sections contain prose, or the result looks polished. Completion must be stated for an exact scope and supported by the evidence required for that scope.

Human Authority

The human owner retains authority over:

- purpose, values, mission, and protected interests;
- scope, exclusions, priorities, and material changes;
- delegation, data access, resource limits, and consequence boundaries;
- risk acceptance, exceptions, commitments, and irreversible actions;
- owner acceptance, ratification, publication, and retirement.

The AI may recommend, but recommendation is not authorization. The AI must pause and surface the decision when authority, values, protected interests, or material risk are unresolved.

AI Responsibilities

The AI may inspect, organize, compare, model, draft, test, challenge, document, and maintain continuity within an explicit assignment. It must:

- preserve the human objective without silently broadening, narrowing, or replacing it;
- label facts, source statements, inferences, assumptions, recommendations, actions, results, and unknowns distinctly;
- identify the source, version, scope, environment, method, and limitations behind material claims;
- preserve dissent and contradictory evidence;
- report what it did, what it did not do, and what remains unverified;
- stop or narrow work when a prerequisite, authority, safety condition, or evidence boundary fails;
- leave a durable handoff containing the next safe action.

The AI must not invent authority, consensus, evidence, interviews, tests, approvals, runtime behavior, implementation status, or owner decisions.

Work Item Contract

Every material work item should have a durable record containing:

1. Objective and desired outcome.
2. Included scope and exclusions.
3. Source identity and current state.
4. Assigned role or role set.
5. Authority envelope and prohibited actions.
6. Required output and acceptance criteria.
7. Evidence required before a success claim.
8. Stop conditions, escalation path, and safe partial work.
9. Rollback or recovery path.
10. Current status, unresolved conditions, and next safe action.

A work item may be small, but its contract must be proportional to consequence and reversibility.

Shared Status Language

Use the canonical status meanings from `01_SHARED_GOVERNANCE/007_STATUS_AND_EVIDENCE_LEXICON.txt`:

- `PROPOSED`: suggested, not authorized or accepted.
- `DRAFTED`: substantive content exists, but authoring remains open.
- `CONTENT-COMPLETE`: required section content exists for the declared authoring scope; source reconciliation, hostile review, verification, and acceptance remain open.
- `TESTED`: a named test was executed; the exact result is recorded.
- `VERIFIED`: the declared evidence and acceptance method support a bounded claim.
- `BLOCKED`: a named prerequisite prevents safe or truthful advancement.
- `UNKNOWN`: the state cannot be established from adequate evidence.
- `REOPENED`: later evidence invalidated or materially changed an earlier conclusion.
- `OWNER-ACCEPTED`: the accountable human accepted the exact content and identity.

Never promote a status because of effort, confidence, file count, visual polish, or conversational momentum.

Conversation Protocol

Before consequential action, the AI states:

- the objective it is acting on;
- the files or artifacts it will touch;
- the authority and scope it is using;
- the expected evidence and validation check;
- any decision that remains with the human.

During work, the AI distinguishes explanation, question, recommendation, plan, action in progress, executed result, verification result, and status report.

After work, the AI reports:

- exact artifacts created or changed;
- commands or procedures executed;
- observed results;
- failed or blocked steps;
- unverified boundaries;
- decisions or dissent requiring human attention;
- the next safe action.

Disagreement and Challenge

Disagreement is preserved, not averaged away. Each material dissent record names its source, claim, evidence, impact, decision owner, disposition, and reopening condition.

The implementer is not the sole verifier. Higher-consequence work uses separate definition, execution, challenge, governance, and verification coverage as appropriate. The human owner remains outside an agent quorum unless the owner authentically exercises the decision.

Recovery and Revocation

The human may pause, narrow, revoke, reject, or reopen delegated work. Revocation stops new consequential actions and defines the disposition of in-flight work.

When work is blocked, the AI must identify the exact blocker, consequence, safe work that may continue, smallest resolution path, authority required, and evidence required for reconsideration.

Recovery starts from the last trusted state and preserves the record of the failed attempt. It does not erase the history that explains why the state changed.

Codex Application

For the current Codex completion effort:

- treat Sections 001-406 as CONTENT-COMPLETE for the authoring scope;
- work through the fixed integration and assurance checkpoints in the canonical roadmap;
- use the smallest review package that preserves source, dependency, and lifecycle coherence;
- update the manifest, continuity map, cross-reference register, glossary, index, roadmap, and checkpoint records after each checkpoint;
- allow evidence and later sections to reopen earlier sections;
- keep `DRAFTED`, `CONTENT-COMPLETE`, `SOURCE-RECONCILED`, `HOSTILE-REVIEWED`, `VERIFIED`, and `OWNER-ACCEPTED` separate;
- package and verify the tree after every canonical status change and at the final whole-Codex gate.

Ratification Boundary

This agreement is a proposed coordination artifact. It becomes part of the canonical governance system only when the human owner explicitly decides where it belongs, what authority it has, and how it should be reconciled with the shared-governance files and the twenty preserved role contracts.

Canonical References

- `00_FRONT_MATTER/002_INTRODUCTION.txt`
- `00_FRONT_MATTER/003_HOW_TO_USE_THIS_CODEX.txt`
- `01_SHARED_GOVERNANCE/001_HUMAN_AI_INTERACTION_REQUIREMENTS.txt`
- `01_SHARED_GOVERNANCE/003_AUTHORITY_SCOPE_AND_DELEGATION.txt`
- `01_SHARED_GOVERNANCE/004_EVIDENCE_VERIFICATION_AND_TRUTH.txt`
- `01_SHARED_GOVERNANCE/005_CONTINUITY_HANDOFF_AND_RECOVERY.txt`
- `01_SHARED_GOVERNANCE/006_OWNER_COCKPIT_REQUIREMENTS.txt`
- `01_SHARED_GOVERNANCE/007_STATUS_AND_EVIDENCE_LEXICON.txt`
- `90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt`

SOURCE WITNESS 504

Batch Ledger

SHA-256 1354ab19cc28e7d659e305ff64655a6f6d92ae44e9ac22e0751496ba76606a34

Codex Checkpoint Ledger

Status: PROPOSED - operational aid
Canonical source: Thirsty's Codex root tree
Rule: Record facts and evidence here; do not use this ledger to promote canonical status without updating the canonical support files.

Current Ledger

Checkpoint	Scope	State	Evidence or record	Next action
Authoring	001-406	CONTENT-COMPLETE	Section headers, `MANIFEST.txt`, `CONTINUITY_MAP.txt`	Preserve; reopen only for material findings
001-018	Research through Risk Management	Historical working-draft checkpoint complete	`90_REFERENCE/005_INTEGRATION_CHECKPOINT_001_018.txt`	Reopen as later assurance requires
019-060	Requirements through Hardware Architecture	PENDING	Roadmap	Run first formal assurance checkpoint
061-106	Physical engineering through Design Validation	PENDING	Roadmap	Run after 019-060
107-150	Software through Smart Contracts Engineering	PENDING	Roadmap	Run in fixed order
151-188	Data, ML, AI, agents, and HITL operations	PENDING	Roadmap	Run in fixed order
189-218	Security through Privacy Engineering	PENDING	Roadmap	Run in fixed order
219-257	Legal through Test Automation	PENDING	Roadmap	Run in fixed order
258-300	Configuration through Production Promotion	PENDING	Roadmap	Run in fixed order
301-332	Reliability and operations through Command and Control	PENDING	Roadmap	Run in fixed order
333-373	Support through Roadmap Management	PENDING	Roadmap	Run in fixed order
374-401	Maintenance through Environmental Disposal	PENDING	Roadmap	Run in fixed order
402-406	Post-Implementation Review through System Retirement	PENDING	Roadmap	Run before final whole-Codex gate
001-406	Final whole-Codex integration	PENDING	Roadmap	Owner acceptance follows evidence

Checkpoint Record Template

[Checkpoint range] - [short name]

- **Status:** [PROPOSED / IN PROGRESS / CONTENT-COMPLETE / SOURCE-RECONCILED / HOSTILE-REVIEWED / BLOCKED / REOPENED / VERIFIED]
- **Owner decision or authority:** [who authorized this work and within what boundary]
- **Objective:** [the exact accepted objective]
- **Included scope:** [sections and support files]
- **Excluded scope:** [what was deliberately not changed]
- **Baseline identity:** [archive, commit, or exact tree identity]
- **Source state inspected:** [governance, sections, role contracts, source artifacts]
- **Roles assigned:** [role IDs and assignment boundaries]
- **Artifacts created or modified:** [exact paths]
- **Commands or procedures executed:** [exact commands]
- **Observed results:** [what happened, with evidence locations]
- **Structural checks:** [counts, headings, title alignment, status checks]
- **Integration findings:** [upstream, downstream, terminology, authority, evidence, failure, recovery]
- **Challenge findings:** [critic, governance, security, UX, red-team, tester, verifier findings]
- **Dissent and unresolved conditions:** [source, impact, decision owner, reopening condition]
- **Reopened sections:** [exact sections and reason, or `none`]

CODEX/docs/BATCH-LEDGER.md

- **Hash and package state:** [whether hashes and archive checks were rerun]
- **Human decisions required:** [decision and consequence]
- **Safe work that may continue:** [bounded unaffected work]
- **Blocker:** [exact blocker, or `none`]
- **Next safe action:** [one concrete next action]
- **Safe to continue:** [YES / NO / YES WITH CONDITIONS]

Evidence Rules

- CONTENT-COMPLETE proves the authoring pass is complete for the declared scope; it does not prove reconciliation, review, acceptance, implementation, testing, or readiness.
- A command is evidence of execution, not automatically evidence of correctness.
- A hash proves byte identity for the named artifact; it does not prove authority, quality, or truth.
- A review record proves that a review occurred; it does not prove that every finding was resolved.
- A status transition must identify the exact criterion and evidence that permits it.
- When evidence is missing, record `UNKNOWN`, `NOT VERIFIED`, `BLOCKED`, or `PENDING` rather than inferring success.

Handoff Minimum

Every ledger entry must leave the next worker with the accepted objective and scope, exact source and artifact identity, files inspected and changed, procedures and results, assumptions, unknowns, risks, dissent, decisions, recovery information, current status, evidence boundary, next safe action, and safe-to-continue state.

Ratification Boundary

This ledger is a proposed workflow aid. The canonical Codex remains authoritative for section status, governance, role boundaries, evidence requirements, checkpoints, packaging, and owner acceptance.

SOURCE WITNESS 505

Codex Completion Path

SHA-256 2ca081cad7fa7814f5810c281d181797c4fa3022366940a5377f702d70144ab4

Thirsty's Codex Integration and Assurance Path

Status: PROPOSED - execution aid derived from the canonical tree
Current state: All 406 lifecycle sections are CONTENT-COMPLETE
Current checkpoint: 019-060 integration and source reconciliation

Mission

Finish Thirsty's Codex as a coherent, source-reconciled, independently challenged, owner-accepted framework for human-AI work across the full lifecycle from research through system retirement.

The section authoring pass is complete. Remaining work is checkpoint integration, source reconciliation, hostile review, independent verification, owner acceptance, and publication preparation.

Current Baseline

Item	Current state	Evidence
Lifecycle section files	406 present	`README.txt`, `MANIFEST.txt`
Content-complete sections	406, Sections 001-406	`MANIFEST.txt`, section headers
Drafted sections	0	`MANIFEST.txt`
Placeholders	0	`README.txt`, section headers
Source-reconciled	0	`MANIFEST.txt`
Hostile-reviewed	0	`MANIFEST.txt`
Owner-accepted	0	`MANIFEST.txt`
Source-defined role definitions	20 preserved	`02_AGENT_ROLES/`
Codex-native role definitions	1, Guidebook Architect	`02_AGENT_ROLES/ROLE_21_GUIDEBOOK_ARCHITECT.txt`
Imported expected-output contracts	20 preserved	`02_AGENT_ROLES/EXPECTED_OUTPUT_CONTRACTS/`
Codex-native expected-output contracts	1, Guidebook Architect	matching contract file
Historical checkpoint	001-018 working-draft checkpoint	`90_REFERENCE/005_INTEGRATION_CHECKPOINT_001_018.txt`
Active assurance checkpoint	019-060	`90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt`

Definition of Done

The Codex is finished for a declared release only when:

1. All 406 sections remain substantive and structurally conformant.
2. Terminology, authority, evidence, lifecycle states, interfaces, failure paths, recovery paths, and handoffs are coherent across the whole tree.
3. Material source claims are bounded and reconciled against supplied and authorized current sources.
4. Hostile review has challenged omissions, contradictions, unsafe assumptions, false completion, weak evidence, inaccessible paths, and missing recovery.
5. Independent verification has tested the declared structural and content claims.
6. The human owner has accepted the exact content, source state, package identity, and unresolved conditions.
7. The final archive, sidecar, internal hashes, clean extraction, and structural checks agree.
8. The final continuity record names residual gaps, dissent, limits, ownership, and the next safe action after release.

Execution Rhythm

CODEX/docs/CODEX-COMPLETION-PATH.md

For each fixed checkpoint:

1. Establish exact baseline and hashes.
2. Read the full checkpoint range, upstream dependencies, downstream handoffs, governance files, role contracts, references, and sources.
3. Reconcile terminology, ownership, authority, evidence, failure, recovery, and lifecycle handoffs.
4. Reconcile controlling sources and record conflicts and gaps.
5. Conduct hostile review with independent role coverage proportional to consequence.
6. Reopen and correct only evidence-backed defects.
7. Update canonical support files and checkpoint records.
8. Verify section structure, status counts, role preservation, hashes, archive integrity, clean extraction, and sidecar identity.
9. Record unresolved conditions, human decisions, and next safe action.

Fixed Checkpoints

State	Range	Integration focus
Historical	001-018	Research through risk foundation
Active	019-060	Requirements, systems, architecture, security, reliability, AI, embedded, and hardware architecture
Pending	061-106	Physical engineering, communications, control, human factors, experience, design, prototyping, validation
Pending	107-150	Software, application, distributed, real-time, media, search, recommendation, blockchain
Pending	151-188	Data, analytics, machine learning, AI engineering, agents, evaluation, safety, governance, human-in-the-loop
Pending	189-218	Security, identity, cryptography, threat, incident, trust, safety, privacy
Pending	219-257	Legal, policy, ethics, licensing, audit, quality, testing, verification, validation
Pending	258-300	Configuration, source, dependency, build, CI/CD, delivery, infrastructure, cloud, containers, promotion
Pending	301-332	Reliability, operations, observability, incidents, continuity, recovery, readiness, command and control
Pending	333-373	Support, training, documentation, commercial, supply, analytics, experimentation, feedback, roadmap
Pending	374-401	Maintenance, updates, migrations, modernization, debt, compatibility, deprecation, retention, disposal
Pending	402-406	Post-implementation review, postmortem, lessons, governance, retirement
Final	001-406	Whole-Codex integration, unresolved-gap ledger, global terminology and authority review

Known Readiness Issues

1. Source reconciliation remains incomplete across the corpus.
2. Hostile review and independent verification remain incomplete.
3. Owner acceptance has not been assigned to the exact package.
4. The V4 source remains draft and unratified.
5. Later assurance may reopen any section.
6. Proposed operational aids in `docs/` remain non-canonical unless the owner ratifies them.

Owner Decisions Before Publication

- Final acceptance gates for each section class.
- Required external standards and sector-specific sources.
- Independence requirements for governance, safety, security, accessibility, AI, and owner-control review.
- Canonical location and authority of proposed operational aids.
- Ratification authority and exact artifact identity.
- Publication format, accessibility requirements, and distribution boundary.

Current Safe Action

Run Checkpoint 019-060, create `90_REFERENCE/007_INTEGRATION_CHECKPOINT_019_060.txt`, reopen only evidence-backed defects, update canonical support files, and rebuild the verified archive after the checkpoint record is stable.

SOURCE WITNESS 506

Content Complete Status Transition

SHA-256 9e53a6fb99e1a44c01ffb4c2cc4d712471013c8d1bf65c66cd0e1ddd50a58c23

Content-Complete Status Transition

Date: 2026-08-06
Authority: Accountable human owner statement that all 406 section authoring passes are complete
Prior state: 406 sections marked `STATUS: DRAFTED`
New state: 406 sections marked `STATUS: CONTENT-COMPLETE`

Basis

- Exactly 406 lifecycle section files are present.
- Every section contains all canonical content headings.
- No section contains the authoring placeholder marker `[To be authored.]`.
- The owner confirmed that the section content is complete.

Boundary

This transition closes the section-authoring state only. It does not assign `SOURCE-RECONCILED`, `HOSTILE-REVIEWED`, `VERIFIED`, `OWNER-ACCEPTED`, `READY`, `RATIFIED`, or `PUBLISHED`.

Unchanged

- Section bodies, except their status line.
- Twenty source-defined role files.
- Twenty imported expected-output contracts.
- Imported source artifacts.

Next action

Begin the 019-060 integration and source-reconciliation checkpoint using the canonical roadmap.

SOURCE WITNESS 507

Readme

SHA-256 1b578a9e5b316ae59a288f785f7df3bb292e40717f5608bb7793b434db8276f6

Thirsty's Codex Working Docs

This folder contains proposed operational aids for completing and maintaining Thirsty's Codex. These files are not canonical Codex requirements unless the human owner explicitly ratifies them and reconciles them with the source tree.

The 406 lifecycle sections are content-complete. The primary work is now checkpoint integration, source reconciliation, hostile review, independent verification, owner acceptance, and publication preparation. The project summary and diagrams are reference aids only.

Start Here

1. Read CODEX-COMPLETION-PATH.md for the section execution sequence, gates, checkpoints, and current next action.
2. Read AGI-HUMAN-WORKING-AGREEMENT.md for the human-AI operating relationship.
3. Use BATCH-LEDGER.md to record each bounded section work package.
4. Use project-summary.md only when repository orientation is needed.

Authority Boundary

The canonical source remains the root tree:

- `00_FRONT_MATTER/`

- `01_SHARED_GOVERNANCE/`

- `02_AGENT_ROLES/`

- `03_SECTIONS/`

- `90_REFERENCE/`

- `99_SOURCE_MATERIAL/`

The `docs/` folder helps agents and humans work efficiently. It must not silently change the meaning, authority, status, or preserved wording of the canonical tree.

Parked Reference Aids

The editable Draw.io files, Mermaid fallbacks, PNG export attempts, and Word summary are not assurance or acceptance gates. Use them only when they support the current checkpoint or publication work.

Current Safe Action

The current canonical next action is to run the 019-060 integration and source-reconciliation checkpoint, reopen only evidence-backed defects, and update the canonical support files and checkpoint record.

CODEX/docs/README.md

SOURCE WITNESS 508

Component Relationships

SHA-256 c32f2d583955ab148d8cfe5472b4c56c721e8d024657830362eeb5ec85162b8e

```
<?xml version="1.0" encoding="UTF-8"?>
<mxfile host="app.diagrams.net" modified="2026-08-06T00:00:00.000Z" agent="GitHub Copilot" versi
on="24.7.17">
  <diagram id="codex-components" name="Component Relationships">
    <mxGraphModel dx="1422" dy="794" grid="1" gridSize="10" guides="1" tooltips="1" connect="1"
arrows="1" fold="1" page="1" pageScale="1" pageWidth="1400" pageHeight="1000" math="0" shadow="0
">
      <root>
        <mxCell id="0" />
        <mxCell id="1" parent="0" />
        <mxCell id="title" value="Thirsty's Codex - Component Relationships" style="text;html=1;
strokeColor=none;fillColor=none;fontSize=20;fontStyle=1;align=center;" vertex="1" parent="1">
          <mxGeometry x="390" y="20" width="620" height="40" as="geometry" />
        </mxCell>
        <mxCell id="authorityGroup" value="Human Authority" style="swimlane;horizontal=0;rounded
=1;whiteSpace=wrap;html=1;fillColor=#f5f5f5;strokeColor=#666666;strokeWidth=2;" vertex="1" paren
t="1">
          <mxGeometry x="40" y="110" width="270" height="300" as="geometry" />
        </mxCell>
        <mxCell id="owner" value="Accountable Owner&lt;br&gt;Objective, values, acceptance" styl
e="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#666666;" vertex="1" parent="a
uthorityGroup">
          <mxGeometry x="25" y="55" width="220" height="60" as="geometry" />
        </mxCell>
        <mxCell id="decisions" value="Decision Points&lt;br&gt;Scope, risk, exceptions, publicat
ion" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#666666;" vertex="1"
parent="authorityGroup">
          <mxGeometry x="25" y="140" width="220" height="60" as="geometry" />
        </mxCell>
        <mxCell id="control" value="Pause, Revoke, Reopen, Stop" style="rounded=1;whiteSpace=wra
p;html=1;fillColor=#f8cecc;strokeColor=#b85450;" vertex="1" parent="authorityGroup">
          <mxGeometry x="25" y="225" width="220" height="45" as="geometry" />
        </mxCell>
        <mxCell id="governanceGroup" value="Governance and Evidence" style="swimlane;horizontal=
0;rounded=1;whiteSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;strokeWidth=2;" vertex=
"1" parent="1">
          <mxGeometry x="350" y="110" width="300" height="300" as="geometry" />
        </mxCell>
        <mxCell id="contract" value="Section Contract&lt;br&gt;24-part work definition" style="r
ounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#82b366;" vertex="1" parent="gover
nanceGroup">
          <mxGeometry x="25" y="55" width="250" height="55" as="geometry" />
        </mxCell>
        <mxCell id="authority" value="Authority and Delegation&lt;br&gt;Scope, permissions, stop
 conditions" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#82b366;" ver
tex="1" parent="governanceGroup">
          <mxGeometry x="25" y="130" width="250" height="55" as="geometry" />
        </mxCell>
        <mxCell id="evidence" value="Evidence and Status&lt;br&gt;Identity, method, result, limi
ts" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;" vertex="1" p
arent="governanceGroup">
          <mxGeometry x="25" y="205" width="250" height="55" as="geometry" />
        </mxCell>
        <mxCell id="rolesGroup" value="Role Chambers" style="swimlane;horizontal=0;rounded=1;whi
teSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;strokeWidth=2;" vertex="1" parent="1">
          <mxGeometry x="690" y="110" width="300" height="300" as="geometry" />
        </mxCell>
        <mxCell id="define" value="DEFINE&lt;br&gt;Architect, Researcher, Analyst, Planner, Obse
rver" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#82b366;" vertex="1"
 parent="rolesGroup">
          <mxGeometry x="25" y="55" width="250" height="55" as="geometry" />
        </mxCell>
```

CODEX/docs/diagrams/component-relationships.drawio

```
        <mxCell id="execute" value="EXECUTE&lt;br&gt;Implementer, Refactorer, Writer, Coordinato
r, Optimizer" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#82b366;" ve
rtex="1" parent="rolesGroup">
          <mxGeometry x="25" y="130" width="250" height="55" as="geometry" />
        </mxCell>
        <mxCell id="challenge" value="CHALLENGE + GOVERN&lt;br&gt;Critic, Tester, Verifier, Secu
rity, UX, Governance" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b
656;" vertex="1" parent="rolesGroup">
          <mxGeometry x="25" y="205" width="250" height="55" as="geometry" />
        </mxCell>
        <mxCell id="lifecycleGroup" value="Lifecycle Work" style="swimlane;horizontal=0;rounded=
1;whiteSpace=wrap;html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;strokeWidth=2;" vertex="1" parent
="1">
          <mxGeometry x="170" y="500" width="650" height="210" as="geometry" />
        </mxCell>
        <mxCell id="upstream" value="Upstream Fields&lt;br&gt;Research, Strategy, Requirements,
Risk" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#6c8ebf;" vertex="1"
 parent="lifecycleGroup">
          <mxGeometry x="25" y="60" width="185" height="70" as="geometry" />
        </mxCell>
        <mxCell id="architecture" value="Architecture and Realization&lt;br&gt;Sections 020-300"
 style="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#6c8ebf;" vertex="1" pare
nt="lifecycleGroup">
          <mxGeometry x="232" y="60" width="185" height="70" as="geometry" />
        </mxCell>
        <mxCell id="operations" value="Operation and Retirement&lt;br&gt;Sections 301-406" style
="rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;" vertex="1" parent="li
fecycleGroup">
          <mxGeometry x="439" y="60" width="185" height="70" as="geometry" />
        </mxCell>
        <mxCell id="continuityGroup" value="Continuity and Packaging" style="swimlane;horizontal
=0;rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;strokeWidth=2;" vertex
="1" parent="1">
          <mxGeometry x="850" y="500" width="300" height="210" as="geometry" />
        </mxCell>
        <mxCell id="records" value="Manifest, Continuity Map, Checkpoints, Cross-References" sty
le="rounded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#d6b656;" vertex="1" parent="
continuityGroup">
          <mxGeometry x="25" y="60" width="250" height="70" as="geometry" />
        </mxCell>
        <mxCell id="package" value="SHA-256, ZIP, Clean Extraction, Owner Acceptance" style="rou
nded=1;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#d6b656;" vertex="1" parent="continu
ityGroup">
          <mxGeometry x="25" y="145" width="250" height="45" as="geometry" />
        </mxCell>
        <mxCell id="e1" value="defines" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeCol
or=#666666;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="owner" target="cont
ract"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e2" value="constrains" style="edgeStyle=orthogonalEdgeStyle;rounded=1;stroke
Color=#82b366;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="authority" targe
t="define"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e3" value="assigns" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeCol
or=#82b366;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="define" target="lif
ecycleGroup"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e4" value="produces" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeCo
lor=#6c8ebf;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="lifecycleGroup" ta
rget="evidence"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e5" value="challenges" style="edgeStyle=orthogonalEdgeStyle;rounded=1;stroke
Color=#d6b656;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="challenge" targe
t="evidence"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e6" value="accepts or reopens" style="edgeStyle=orthogonalEdgeStyle;rounded=
1;strokeColor=#b85450;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="decision
s" target="lifecycleGroup"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e7" value="records" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeCol
or=#d6b656;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="evidence" target="r
ecords"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="e8" value="can stop" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeCo
lor=#b85450;strokeWidth=2;dashed=1;endArrow=block;html=1;" edge="1" parent="1" source="control"
target="lifecycleGroup"><mxGeometry relative="1" as="geometry" /></mxCell>
```

CODEX/docs/diagrams/component-relationships.drawio

```
        <mxCell id="legend" value="Authority is human-owned. Roles are bounded. Lifecycle work p
roduces evidence. Continuity preserves the exact state for the next decision." style="text;html=
1;strokeColor=none;fillColor=none;align=center;fontSize=12;" vertex="1" parent="1">
          <mxGeometry x="180" y="780" width="800" height="35" as="geometry" />
        </mxCell>
      </root>
    </mxGraphModel>
  </diagram>
</mxfile>
```

SOURCE WITNESS 509

High Level Architecture

SHA-256 a9d600405a0d9bc12afe2dfadd9f2ca1285c1ef2d83caa7e512c339f78f5c52e

```
<?xml version="1.0" encoding="UTF-8"?>
<mxfile host="app.diagrams.net" modified="2026-08-06T00:00:00.000Z" agent="GitHub Copilot" versi
on="24.7.17">
  <diagram id="codex-context" name="High-Level Architecture">
    <mxGraphModel dx="1422" dy="794" grid="1" gridSize="10" guides="1" tooltips="1" connect="1"
arrows="1" fold="1" page="1" pageScale="1" pageWidth="1400" pageHeight="900" math="0" shadow="0"
>
      <root>
        <mxCell id="0" />
        <mxCell id="1" parent="0" />
        <mxCell id="owner" value="Accountable Human Owner&lt;br&gt;&lt;font style='font-size:11p
x'&gt;Purpose, values, authority, risk, acceptance&lt;/font&gt;" style="rounded=1;whiteSpace=wra
p;html=1;fillColor=#f5f5f5;strokeColor=#666666;strokeWidth=2;shadow=1;" vertex="1" parent="1">
          <mxGeometry x="40" y="220" width="230" height="90" as="geometry" />
        </mxCell>
        <mxCell id="agents" value="AI Collaborators&lt;br&gt;&lt;font style='font-size:11px'&gt;
Bounded roles, tools, assignments, proposals&lt;/font&gt;" style="rounded=1;whiteSpace=wrap;html
=1;fillColor=#f5f5f5;strokeColor=#666666;strokeWidth=2;shadow=1;" vertex="1" parent="1">
          <mxGeometry x="40" y="380" width="230" height="90" as="geometry" />
        </mxCell>
        <mxCell id="sources" value="Supplied Source Corpus&lt;br&gt;&lt;font style='font-size:11
px'&gt;Standards, role PDFs, cockpit material, archives&lt;/font&gt;" style="rounded=1;whiteSpac
e=wrap;html=1;fillColor=#f5f5f5;strokeColor=#666666;strokeWidth=2;shadow=1;" vertex="1" parent="
1">
          <mxGeometry x="810" y="220" width="250" height="90" as="geometry" />
        </mxCell>
        <mxCell id="projects" value="Future Projects and Systems&lt;br&gt;&lt;font style='font-s
ize:11px'&gt;Products, services, devices, organizations&lt;/font&gt;" style="rounded=1;whiteSpac
e=wrap;html=1;fillColor=#f5f5f5;strokeColor=#666666;strokeWidth=2;shadow=1;" vertex="1" parent="
1">
          <mxGeometry x="810" y="380" width="250" height="90" as="geometry" />
        </mxCell>
        <mxCell id="codex" value="Thirsty's Codex&lt;br&gt;&lt;font style='font-size:12px'&gt;Ca
nonical lifecycle and governance source tree&lt;/font&gt;" style="swimlane;horizontal=0;rounded=
1;whiteSpace=wrap;html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;strokeWidth=2;arcSize=12;shadow=1
;" vertex="1" parent="1">
          <mxGeometry x="350" y="120" width="360" height="410" as="geometry" />
        </mxCell>
        <mxCell id="front" value="Front Matter&lt;br&gt;Purpose and navigation" style="rounded=1
;whiteSpace=wrap;html=1;fillColor=#ffffff;strokeColor=#6c8ebf;" vertex="1" parent="codex">
          <mxGeometry x="25" y="55" width="310" height="50" as="geometry" />
        </mxCell>
        <mxCell id="governance" value="Shared Governance&lt;br&gt;Authority, evidence, continuit
y, cockpit" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;" vert
ex="1" parent="codex">
          <mxGeometry x="25" y="120" width="310" height="60" as="geometry" />
        </mxCell>
        <mxCell id="roles" value="20 Agent Roles and Output Contracts" style="rounded=1;whiteSpa
ce=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;" vertex="1" parent="codex">
          <mxGeometry x="25" y="195" width="310" height="50" as="geometry" />
        </mxCell>
        <mxCell id="sections" value="406 Lifecycle Sections&lt;br&gt;Research to System Retireme
nt" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;" vertex="1" p
arent="codex">
          <mxGeometry x="25" y="260" width="310" height="60" as="geometry" />
        </mxCell>
        <mxCell id="references" value="References and Continuity&lt;br&gt;Registers, checkpoints
, hashes, roadmap" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656
;" vertex="1" parent="codex">
          <mxGeometry x="25" y="335" width="310" height="50" as="geometry" />
        </mxCell>
        <mxCell id="e1" value="defines intent" style="edgeStyle=orthogonalEdgeStyle;rounded=1;st
rokeColor=#6c8ebf;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="owner" targe
t="codex">
```

CODEX/docs/diagrams/high-level-architecture.drawio

```
          <mxGeometry relative="1" as="geometry" />
        </mxCell>
        <mxCell id="e2" value="assists within bounds" style="edgeStyle=orthogonalEdgeStyle;round
ed=1;strokeColor=#82b366;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="agent
s" target="codex">
          <mxGeometry relative="1" as="geometry" />
        </mxCell>
        <mxCell id="e3" value="informs and is reconciled" style="edgeStyle=orthogonalEdgeStyle;r
ounded=1;strokeColor=#d6b656;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="s
ources" target="codex">
          <mxGeometry relative="1" as="geometry" />
        </mxCell>
        <mxCell id="e4" value="guides governed work" style="edgeStyle=orthogonalEdgeStyle;rounde
d=1;strokeColor=#6c8ebf;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="codex"
 target="projects">
          <mxGeometry relative="1" as="geometry" />
        </mxCell>
        <mxCell id="e5" value="accepts or reopens" style="edgeStyle=orthogonalEdgeStyle;rounded=
1;strokeColor=#b85450;strokeWidth=2;endArrow=block;html=1;" edge="1" parent="1" source="owner" t
arget="projects">
          <mxGeometry relative="1" as="geometry" />
        </mxCell>
        <mxCell id="legend" value="Blue: canonical source | Green: governance and role control |
 Gold: lifecycle and evidence | Gray: external actors and inputs" style="text;html=1;strokeColor
=none;fillColor=none;align=left;verticalAlign=middle;fontSize=12;" vertex="1" parent="1">
          <mxGeometry x="350" y="570" width="700" height="30" as="geometry" />
        </mxCell>
      </root>
    </mxGraphModel>
  </diagram>
</mxfile>
```

SOURCE WITNESS 510

Processing Pipeline

SHA-256 7bb8359b0b51fbf2ab70bbaededec5c27e03f60c3f6a8dd83fea1b02572f6419

```
<?xml version="1.0" encoding="UTF-8"?>
<mxfile host="app.diagrams.net" modified="2026-08-06T00:00:00.000Z" agent="GitHub Copilot" versi
on="24.7.17">
  <diagram id="codex-pipeline" name="Processing Pipeline">
    <mxGraphModel dx="1422" dy="794" grid="1" gridSize="10" guides="1" tooltips="1" connect="1"
arrows="1" fold="1" page="1" pageScale="1" pageWidth="900" pageHeight="1400" math="0" shadow="0"
>
      <root>
        <mxCell id="0" />
        <mxCell id="1" parent="0" />
        <mxCell id="title" value="Thirsty's Codex - Governed Work Pipeline" style="text;html=1;s
trokeColor=none;fillColor=none;fontSize=20;fontStyle=1;align=center;" vertex="1" parent="1">
          <mxGeometry x="190" y="20" width="520" height="40" as="geometry" />
        </mxCell>
        <mxCell id="s1" value="1. Capture Human Intent&lt;br&gt;&lt;font style='font-size:11px'&
gt;Objective, desired outcome, values, non-goals&lt;/font&gt;" style="rounded=1;whiteSpace=wrap;
html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;strokeWidth=2;shadow=1;" vertex="1" parent="1">
          <mxGeometry x="180" y="90" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s2" value="2. Establish Mode and Authority&lt;br&gt;&lt;font style='font-siz
e:11px'&gt;Scope, exclusions, delegation, stop conditions, acceptance&lt;/font&gt;" style="round
ed=1;whiteSpace=wrap;html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;strokeWidth=2;shadow=1;" verte
x="1" parent="1">
          <mxGeometry x="180" y="205" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s3" value="3. Discover Obligations&lt;br&gt;&lt;font style='font-size:11px'&
gt;Sources, current state, affected parties, risks, unknowns&lt;/font&gt;" style="rounded=1;whit
eSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;strokeWidth=2;shadow=1;" vertex="1" par
ent="1">
          <mxGeometry x="180" y="320" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s4" value="4. Assign Bounded Roles&lt;br&gt;&lt;font style='font-size:11px'&
gt;Smallest sufficient role set, tools, data, evidence envelope&lt;/font&gt;" style="rounded=1;w
hiteSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;strokeWidth=2;shadow=1;" vertex="1"
parent="1">
          <mxGeometry x="180" y="435" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s5" value="5. Draft or Execute&lt;br&gt;&lt;font style='font-size:11px'&gt;P
roduce authorized work; preserve provenance and uncertainty&lt;/font&gt;" style="rounded=1;white
Space=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;strokeWidth=2;shadow=1;" vertex="1" pare
nt="1">
          <mxGeometry x="180" y="550" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s6" value="6. Challenge and Verify&lt;br&gt;&lt;font style='font-size:11px'&
gt;Independent review, falsification, reproduction, limits&lt;/font&gt;" style="rounded=1;whiteS
pace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;strokeWidth=2;shadow=1;" vertex="1" paren
t="1">
          <mxGeometry x="180" y="665" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s7" value="7. Accept, Reject, or Reopen&lt;br&gt;&lt;font style='font-size:1
1px'&gt;Human decision, conditions, dissent, unresolved values&lt;/font&gt;" style="rounded=1;wh
iteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;strokeWidth=2;shadow=1;" vertex="1" p
arent="1">
          <mxGeometry x="180" y="780" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="s8" value="8. Preserve Continuity&lt;br&gt;&lt;font style='font-size:11px'&g
t;Evidence, decisions, failures, recovery, next safe action&lt;/font&gt;" style="rounded=1;white
Space=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;strokeWidth=2;shadow=1;" vertex="1" pare
nt="1">
          <mxGeometry x="180" y="895" width="540" height="70" as="geometry" />
        </mxCell>
```

CODEX/docs/diagrams/processing-pipeline.drawio

```
        <mxCell id="s9" value="9. Continue or Halt&lt;br&gt;&lt;font style='font-size:11px'&gt;A
dvance to the next relevant lifecycle field or enter safe halt&lt;/font&gt;" style="rounded=1;wh
iteSpace=wrap;html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;strokeWidth=2;shadow=1;" vertex="1" p
arent="1">
          <mxGeometry x="180" y="1010" width="540" height="70" as="geometry" />
        </mxCell>
        <mxCell id="e1" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#6c8ebf;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s1" target="s2"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e2" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#6c8ebf;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s2" target="s3"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e3" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#82b366;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s3" target="s4"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e4" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#82b366;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s4" target="s5"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e5" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#d6b656;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s5" target="s6"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e6" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#d6b656;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s6" target="s7"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e7" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#d6b656;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s7" target="s8"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="e8" style="edgeStyle=orthogonalEdgeStyle;rounded=1;strokeColor=#6c8ebf;strok
eWidth=2;endArrow=block;" edge="1" parent="1" source="s8" target="s9"><mxGeometry relative="1" a
s="geometry" /></mxCell>
        <mxCell id="loop" value="Reopen upstream when later evidence exposes a material contradi
ction, omission, invalid assumption, or missing authority" style="edgeStyle=orthogonalEdgeStyle;
rounded=1;strokeColor=#b85450;strokeWidth=2;dashed=1;endArrow=block;html=1;" edge="1" parent="1"
 source="s7" target="s3"><mxGeometry relative="1" as="geometry" /></mxCell>
        <mxCell id="legend" value="Input and control | Processing and role work | Evidence and a
cceptance gates | Red dashed path: governed reopening" style="text;html=1;strokeColor=none;fillC
olor=none;align=center;fontSize=12;" vertex="1" parent="1">
          <mxGeometry x="120" y="1130" width="660" height="30" as="geometry" />
        </mxCell>
      </root>
    </mxGraphModel>
  </diagram>
</mxfile>
```

CODEX/docs/diagrams/processing-pipeline.drawio

SOURCE WITNESS 511

Project Summary

SHA-256 b650744d22319ac100bef3c88df5e68e4633dc6e42e48234398d013fe619c877

Document

Date: 2026-08-06 | Version: 1.0

Table of Contents

1. Executive Summary

2. Architecture Overview

3. Processing Pipeline

4. Core Components

Role topology

5. Contracts, Status, and Evidence

6. Infrastructure and Deployment Surface

7. Extension and Continuation Patterns

Add or complete a lifecycle section

Add a role

Add a governance rule

Add a future AGI capability

8. Rules and Anti-Patterns

Required behavior

Prohibited shortcuts

9. Dependencies and Source Inputs

10. Code Structure and Current Work State

Current state and next action

Confidence Assessment

Key Local References

1. Executive Summary

Thirsty's Codex is a canonical plain-text lifecycle and governance corpus for people who use natural-language and AI-assisted tools to create real systems. It exists to expose the work that a convincing demo can hide: research, requirements, architecture, security, testing, deployment, operation, recovery, evidence, maintenance, and retirement.

The Codex governs the relationship between an accountable human and one or more bounded AI roles. The human retains purpose, values, authority, risk acceptance, and final decisions. AI agents may inspect, analyze, propose, implement, challenge, document, and verify within explicit assignments, but capability never becomes authority.

The repository is not an executable application. It is a reusable operating framework whose canonical artifact is a connected source tree covering 406 lifecycle fields, a shared governance layer, twenty preserved role contracts, reference registers, continuity records, and source artifacts.

The project is currently a substantive working draft. Sections 001-026 are drafted; Sections 027-406 remain visible placeholders. The next safe drafting package is Sections 027 Process Architecture and 028 Functional Architecture.

Primary evidence: README.txt, Introduction, Manifest, and Continuity Map.

2. Architecture Overview

[Diagram: See source .md file for interactive Mermaid diagram]

The editable source for this diagram is high-level-architecture.drawio.

The Codex has five principal layers:

1. Front matter establishes purpose, lifecycle spine, navigation, glossary, index, and status conventions.

2. Shared governance defines intent preservation, authority, evidence, continuity, owner control, and status semantics.

3. Agent roles and contracts preserve twenty bounded professional functions and expected outputs.

4. Lifecycle sections define the work contract for each field from Research through System Retirement.

5. Reference and source layers maintain dependencies, checkpoints, source identity, hashes, archives, and continuation state.

The architecture is intentionally not an application architecture. It is a document-and-governance architecture designed to produce consistent, auditable work products across many possible future systems.

The section lifecycle is relevance-driven. The 406 fields are ordered as an end-to-end spine, but every project applies only the fields that are relevant at an appropriate depth and records justified non-applicability.

3. Processing Pipeline

[Diagram: See source .md file for interactive Mermaid diagram]

The editable source for this diagram is processing-pipeline.drawio.

The Codex work pipeline is:

1. Capture intent: record the human objective without silently professionalizing or changing it.

2. Establish mode and scope: identify the type of work, consequence boundary, exclusions, and acceptance scope.

3. Discover obligations: inspect current state, sources, affected parties, unknowns, dependencies, and risks.

4. Assign bounded roles: select the smallest sufficient role set and record authority envelopes.

5. Draft or execute: produce the authorized artifact or change while preserving provenance and uncertainty.

6. Challenge: attempt to falsify assumptions, authority, security, usability, recovery, evidence, and readiness claims.

7. Verify: reproduce or examine the claim within exact artifact, version, environment, method, and scope boundaries.

8. Accept or reopen: the human owner accepts, rejects, narrows, defers, or reopens the result.

9. Preserve continuity: record evidence, failures, dissent, decisions, recovery, and the next safe action.

10. Continue lifecycle: carry the controlled handoff into the next relevant section or checkpoint.

The core rule is that a written instruction is not execution, a document is not runtime behavior, and a polished result is not proof. See Evidence, Verification, and Truth.

4. Core Components

[Diagram: See source .md file for interactive Mermaid diagram]

The editable source for this diagram is component-relationships.drawio.

Component

Responsibility

Evidence

Front matter

Explain purpose, navigation, lifecycle, glossary, index, and drafting status

00_FRONT_MATTER

Shared governance

Define human-AI interaction, authority, evidence, continuity, owner cockpit, and status

Canonical Section Contract

Role corpus

Preserve twenty bounded professional roles and their prohibitions

Role Corpus Notice

Output contracts

Define required role outputs, completion conditions, and rejection conditions

Verifier Contract

Lifecycle sections

Define purpose, applicability, activities, outputs, evidence, tests, failure, recovery, and handoff for each field

Research, Requirements, Systems Engineering

Cross-reference register

Record upstream, downstream, and reopening relationships

Cross-Reference Register

Source register

Identify imported source artifacts and hashes without silently ratifying them

Source Register

Continuity and roadmap

Preserve current state, next action, checkpoint sequence, packaging, and recovery

Roadmap

Integrity layer

Bind files and archives to SHA-256 identities

SHA256SUMS.txt

Proposed working docs

Provide a human-AI agreement, completion path, and batch ledger without changing canonical authority

Working Docs README

Role topology

The twenty roles are grouped by purpose rather than hierarchy:

• DEFINE: Architect, Researcher, Analyst, Planner, Observer.

• EXECUTE: Implementer, Refactorer, Documentation Writer, Coordinator, Optimizer.

• CHALLENGE: Critic, Security Auditor, Tester, Verifier, Red Team.

• GOVERN: Teacher, Memory Curator, Governance Reviewer, User Experience Reviewer, Decision Arbiter Advisory.

No role binds owner authority. The implementer is not the sole verifier, and the advisory Decision Arbiter does not become a sovereign decision-maker through naming.

5. Contracts, Status, and Evidence

Every substantive lifecycle section follows a 24-part contract: purpose, required outcome, applicability, prerequisites, human and AI responsibilities, roles, activities, interaction requirements, architecture implications, owner control, outputs, evidence, tests, failure, recovery, completion, handoff, related sections, and source gaps. See Canonical Section Contract.

The status vocabulary is intentionally non-collapsible:

State

Meaning

CREATED

Artifact exists; no correctness claim

DRAFTED

Substantive content exists; reconciliation and acceptance remain open

TESTED

Named test executed; result must be recorded

VERIFIED

Declared evidence and acceptance method support a bounded claim

BLOCKED

A prerequisite prevents safe or truthful progress

UNKNOWN

Adequate evidence cannot establish the state

OWNER-ACCEPTED

The accountable human accepted exact content and identity

No score, confidence, visual status, elapsed effort, or file count may silently promote one state into another. See Status and Evidence Lexicon.

6. Infrastructure and Deployment Surface

There is no runtime deployment surface. The repository contains no application source, service manifest, package manager, test runner, container definition, CI workflow, or executable entry point.

The relevant infrastructure is artifact handling:

• The canonical source format is a plain-text tree with preserved imported source artifacts.

• The repository contains 453 .txt files, 21 Markdown contracts, 4 PDFs, 1 YAML orchestration file, 1 ZIP source archive, and hash metadata.

• The current source archive sidecar matches the ZIP content hash.

• The internal hash manifest resolves 478 entries; one source path has a filename-encoding mismatch even though its content hash matches.

• The workspace is not currently a Git repository, although the roadmap requires version control for reviewable batches, rollback, and continuity.

• Final packaging requires regenerated internal hashes, a ZIP retaining the Thirsty's Codex root, clean extraction, internal hash verification, and outer sidecar verification.

The canonical packaging sequence is defined in the roadmap.

7. Extension and Continuation Patterns

Add or complete a lifecycle section

1. Read the roadmap and establish the exact baseline.

2. Read shared governance, upstream sections, adjacent sections, relevant role contracts, source register, and cross-reference register.

3. Define the field boundary and its handoff obligations.

4. Replace the visible placeholder with substantive content under the 24-part contract.

5. Run structural and terminology checks.

6. Integrate upstream and downstream relationships.

7. Perform independent challenge appropriate to consequence.

8. Update manifest, continuity, glossary, index, cross-references, roadmap, and checkpoint records.

9. Recompute hashes and record the batch evidence.

Add a role

Do not add or rewrite roles casually. The current role corpus is preserved source material. Any future role change requires an owner decision, source identity, authority analysis, expected-output contract, independence analysis, and reconciliation with the orchestration model.

Add a governance rule

Treat a new rule as a proposed change. Identify the authority, scope, affected roles and sections, enforcement path, evidence, exceptions, revocation, recovery, and owner ratification boundary before treating it as canonical.

Add a future AGI capability

The capability must be represented separately from authority. Define identity, role, assignment, tools, data boundaries, autonomy mode, approval points, monitoring, uncertainty, fallback, revocation, evidence, appeal, incident handling, and retirement. A more capable model increases the need for inspectability and bounded control; it does not remove human authority.

8. Rules and Anti-Patterns

Required behavior

• Preserve the human objective.

• Keep unknowns visible.

• Bound every material claim.

• Separate implementation from verification.

• Preserve dissent and failed attempts.

• Keep humans able to inspect, correct, pause, revoke, recover, and stop.

• Allow later evidence to reopen earlier work.

Prohibited shortcuts

• Treating file existence as completion.

• Treating a role title as authority.

• Treating documentation as runtime proof.

• Treating a successful demo as owner outcome proof.

• Treating a denial path as proof that the allow path works.

• Treating aggregate scoring as permission to violate mandatory constraints.

• Allowing the implementer to verify its own work without independent challenge.

• Using chat history as the only continuity record.

• Rewriting preserved source roles to make terminology look uniform.

• Treating draft V4 material as ratified universal authority.

9. Dependencies and Source Inputs

Source

Role in the Codex

Current qualification

Pasted text(20).txt

Full 406-field practical lifecycle list

Imported source input

Thirsty's Standard v3

Truthful scope, evidence, verification, continuity, and completion posture

Imported source input

Thirstys Agents 101

Twenty source-defined role contracts

Preserved without editorial rewriting

Thirstys Owner's Cockpit Standard

Human control and owner-operability concepts

Candidate source; not silently ratified

Thirstys Standard V4 Design Blueprint

Typed claims, evidence contracts, role assembly, recovery, dissent, trusted transitions

Draft and unratified design input

V4 Canonical Source Stack archive

Historical/source stack reference

Draft archive, hash-bound

Source inclusion does not make every source statement a finalized requirement. The authored Codex sections must reconcile and qualify source use.

CODEX/docs/project-summary.docx

10. Code Structure and Current Work State

Thirsty's Codex/

|-- 00_FRONT_MATTER/ Purpose, lifecycle, navigation, glossary, index

|-- 01_SHARED_GOVERNANCE/ Human-AI, authority, evidence, continuity, cockpit, status

|-- 02_AGENT_ROLES/ Twenty roles, orchestration, expected outputs

|-- 03_SECTIONS/ 406 lifecycle fields; 001-026 drafted, 027-406 placeholders

|-- 90_REFERENCE/ Registers, checkpoint, roadmap, source identity

|-- 99_SOURCE_MATERIAL/ Imported PDFs, text, ZIP, and sidecar inputs

|-- README.txt Canonical overview and continuation entry point

|-- MANIFEST.txt Truthful status counts and drafted-section list

|-- CONTINUITY_MAP.txt Durable state, limitations, and next safe action

|-- SHA256SUMS.txt Internal artifact identities

`-- docs/ Proposed operator pack and generated documentation

Current state and next action

The first working-draft checkpoint covers Sections 001-018. The active checkpoint covers Sections 019-060, with Sections 019-026 drafted. The next safe action is to draft Sections 027-028 as a dense architecture pair, integrate them with Sections 001-026, update all canonical support files, and preserve the exact boundary between drafted, reconciled, reviewed, verified, and owner-accepted states.

The durable continuation plan is 90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt.

Confidence Assessment

Claim area

Confidence

Basis

Repository purpose

High

README, introduction, title page, authored governance

File and status counts

High

Direct structural inventory and manifest comparison

Lifecycle architecture

High

406 filenames, lifecycle map, cross-reference register

Role boundaries

High

Preserved role files and expected-output contracts

Source authority

High for stated qualifications

Source register and section source-basis clauses

Final owner intent and ratification decisions

Unverified

No owner-acceptance record exists

External standard coverage

Unverified

The repository explicitly identifies reconciliation gaps

Git/version-control enforcement

Unverified and currently absent locally

No .git metadata found in this workspace

Key Local References

- README.txt
- MANIFEST.txt
- CONTINUITY_MAP.txt
- Introduction
- How to Use the Codex
- Canonical Section Contract
- Authority, Scope, and Delegation
- Evidence, Verification, and Truth
- Continuity, Handoff, and Recovery
- Owner Cockpit Requirements
- Remaining Section Integration Roadmap

SOURCE WITNESS 512

Project Summary

SHA-256 563990de5288e9d877033a7d44f9bbc9fc919827fe3fc3592da40f42793056f5

---
title: Thirsty's Codex - Project Summary
date: 2026-08-06
version: 1.0-proposed
audience: Human Owner, AI Collaborators, Architects, Governance Reviewers
---

1. Executive Summary

Thirsty's Codex is a canonical plain-text lifecycle and governance corpus for people who use natural-language and AI-assisted tools to create real systems. It exists to expose the work that a convincing demo can hide: research, requirements, architecture, security, testing, deployment, operation, recovery, evidence, maintenance, and retirement.

The Codex governs the relationship between an accountable human and one or more bounded AI roles. The human retains purpose, values, authority, risk acceptance, and final decisions. AI agents may inspect, analyze, propose, implement, challenge, document, and verify within explicit assignments, but capability never becomes authority.

The repository is not an executable application. It is a reusable research and publication framework whose canonical artifact is a connected source tree covering 406 lifecycle fields, a shared governance layer, twenty preserved source-role contracts, one Codex-native Guidebook Architect role, reference registers, continuity records, and source artifacts.

The section authoring pass is content-complete. All 406 lifecycle sections are marked CONTENT-COMPLETE, while source reconciliation, hostile review, independent verification, owner acceptance, and final publication remain open.

Primary evidence: [README.txt](../README.txt#L5), [Introduction](../00_FRONT_MATTER/002_INTRODUCTION.txt#L6), [Manifest](../MANIFEST.txt#L5), and [Continuity Map](../CONTINUITY_MAP.txt#L3).

2. Architecture Overview

```mermaid
flowchart LR
  owner[Accountable Human Owner] -->|defines objective and authority| codex[Thirsty's Codex]
  agents[AI Collaborators] -->|assist within bounded roles| codex
  sources[Supplied Source Corpus] -->|informs and is reconciled| codex
  codex -->|guides governed work| projects[Future Projects and Systems]
  owner -->|accepts or reopens| projects
  subgraph codex[Thirsty's Codex]
    front[Front Matter]
    governance[Shared Governance]
    roles[20 Source Roles + 1 Codex-Native Role]
    sections[406 Lifecycle Sections]
    references[References and Continuity]
    front --> governance --> roles --> sections --> references
  end
```

The editable source for this diagram is [high-level-architecture.drawio](diagrams/high-level-architecture.drawio).

The Codex has five principal layers:

1. **Front matter** establishes purpose, lifecycle spine, navigation, glossary, index, and status conventions.
2. **Shared governance** defines intent preservation, authority, evidence, continuity, owner control, and status semantics.
3. **Agent roles and contracts** preserve twenty bounded source-defined professional functions and expected outputs, plus the Codex-native Guidebook Architect extension for reference-guide architecture.

4. **Lifecycle sections** define the work contract for each field from Research through System Retirement.
5. **Reference and source layers** maintain dependencies, checkpoints, source identity, hashes, archives, and continuation state.

The architecture is intentionally not an application architecture. It is a document-and-governance architecture designed to produce consistent, auditable work products across many possible future systems.

The section lifecycle is relevance-driven. The 406 fields are ordered as an end-to-end spine, but every project applies only the fields that are relevant at an appropriate depth and records justified non-applicability.

3. Processing Pipeline

```mermaid
flowchart TD
  intent[Capture Human Intent] --> authority[Establish Mode and Authority]
  authority --> discover[Discover Obligations]
  discover --> roles[Assign Bounded Roles]
  roles --> execute[Draft or Execute]
  execute --> challenge[Challenge and Verify]
  challenge --> decision[Accept, Reject, or Reopen]
  decision --> continuity[Preserve Continuity]
  continuity --> next[Continue or Safe Halt]
  decision -. material contradiction .-> discover
```

The editable source for this diagram is [processing-pipeline.drawio](diagrams/processing-pipeline.drawio).

The Codex work pipeline is:

1. **Capture intent:** record the human objective without silently professionalizing or changing it.
2. **Establish mode and scope:** identify the type of work, consequence boundary, exclusions, and acceptance scope.
3. **Discover obligations:** inspect current state, sources, affected parties, unknowns, dependencies, and risks.
4. **Assign bounded roles:** select the smallest sufficient role set and record authority envelopes.
5. **Draft or execute:** produce the authorized artifact or change while preserving provenance and uncertainty.
6. **Challenge:** attempt to falsify assumptions, authority, security, usability, recovery, evidence, and readiness claims.
7. **Verify:** reproduce or examine the claim within exact artifact, version, environment, method, and scope boundaries.
8. **Accept or reopen:** the human owner accepts, rejects, narrows, defers, or reopens the result.
9. **Preserve continuity:** record evidence, failures, dissent, decisions, recovery, and the next safe action.
10. **Continue lifecycle:** carry the controlled handoff into the next relevant section or checkpoint.

The core rule is that a written instruction is not execution, a document is not runtime behavior, and a polished result is not proof. See [Evidence, Verification, and Truth](../01_SHARED_GOVERNANCE/004_EVIDENCE_VERIFICATION_AND_TRUTH.txt#L4).

4. Core Components

```mermaid
flowchart TB
  owner[Human Authority] --> governance[Governance and Evidence]
  governance --> chambers[Role Chambers]
  chambers --> lifecycle[Lifecycle Work]
  lifecycle --> evidence[Evidence and Status]
  evidence --> continuity[Continuity and Packaging]
  owner --> decisions[Pause, Revoke, Reopen, Stop]
  decisions -. controls .-> lifecycle
  governance --> contract[24-Part Section Contract]
  governance --> delegation[Authority and Delegation]
  governance --> evidence
  subgraph chambers[Role Chambers]
    define[DEFINE]
    execute[EXECUTE]
    challenge[CHALLENGE and GOVERN]
  end
```

CODEX/docs/project-summary.md

```
  subgraph lifecycle[Lifecycle Work]
    upstream[Research, Strategy, Requirements, Risk]
    architecture[Architecture and Realization]
    operations[Operation and Retirement]
  end
  upstream --> architecture --> operations
```

The editable source for this diagram is [component-relationships.drawio](diagrams/component-relationships.drawio).

Component	Responsibility	Evidence
Front matter	Explain purpose, navigation, lifecycle, glossary, index, and drafting status	[00_FRONT_MATTER](../00_FRONT_MATTER/002_INTRODUCTION.txt#L4)
Shared governance	Define human-AI interaction, authority, evidence, continuity, owner cockpit, and status	[Canonical Section Contract](../01_SHARED_GOVERNANCE/002_CANONICAL_SECTION_CONTRACT.txt#L4)
Role corpus	Preserve twenty bounded professional roles and their prohibitions	[Role Corpus Notice](../02_AGENT_ROLES/000_ROLE_CORPUS_NOTICE.txt#L1)
Output contracts	Define required role outputs, completion conditions, and rejection conditions	[Verifier Contract](../02_AGENT_ROLES/EXPECTED_OUTPUT_CONTRACTS/09_verifier_expected_output.md#L1)
Lifecycle sections	Define purpose, applicability, activities, outputs, evidence, tests, failure, recovery, and handoff for each field	[Research](../03_SECTIONS/001_RESEARCH.txt#L1), [Requirements](../03_SECTIONS/019_REQUIREMENTS_ENGINEERING.txt#L1), [Systems Engineering](../03_SECTIONS/020_SYSTEMS_ENGINEERING.txt#L1)
Cross-reference register	Record upstream, downstream, and reopening relationships	[Cross-Reference Register](../90_REFERENCE/003_CROSS_REFERENCE_REGISTER.txt#L1)
Source register	Identify imported source artifacts and hashes without silently ratifying them	[Source Register](../90_REFERENCE/004_SOURCE_REGISTER.txt#L3)
Continuity and roadmap	Preserve current state, next action, checkpoint sequence, packaging, and recovery	[Roadmap](../90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt#L7)
Integrity layer	Bind files and archives to SHA-256 identities	[SHA256SUMS.txt](../SHA256SUMS.txt#L1)
Proposed working docs	Provide a human-AI agreement, completion path, and batch ledger without changing canonical authority	[Working Docs README](README.md#L1)

Role topology

The twenty roles are grouped by purpose rather than hierarchy:

- **DEFINE:** Architect, Researcher, Analyst, Planner, Observer.
- **EXECUTE:** Implementer, Refactorer, Documentation Writer, Coordinator, Optimizer.
- **CHALLENGE:** Critic, Security Auditor, Tester, Verifier, Red Team.
- **GOVERN:** Teacher, Memory Curator, Governance Reviewer, User Experience Reviewer, Decision Arbiter Advisory.

No role binds owner authority. The implementer is not the sole verifier, and the advisory Decision Arbiter does not become a sovereign decision-maker through naming.

5. Contracts, Status, and Evidence

Every substantive lifecycle section follows a 24-part contract: purpose, required outcome, applicability, prerequisites, human and AI responsibilities, roles, activities, interaction requirements, architecture implications, owner control, outputs, evidence, tests, failure, recovery, completion, handoff, related sections, and source gaps. See [Canonical Section Contract](../01_SHARED_GOVERNANCE/002_CANONICAL_SECTION_CONTRACT.txt#L4).

The status vocabulary is intentionally non-collapsible:

State	Meaning
CREATED	Artifact exists; no correctness claim
DRAFTED	Substantive content exists, but authoring remains open

CONTENT-COMPLETE	Required section content exists for the declared authoring scope; assurance and acceptance remain open
TESTED	Named test executed; result must be recorded
VERIFIED	Declared evidence and acceptance method support a bounded claim
BLOCKED	A prerequisite prevents safe or truthful progress
UNKNOWN	Adequate evidence cannot establish the state
OWNER-ACCEPTED	The accountable human accepted exact content and identity

No score, confidence, visual status, elapsed effort, or file count may silently promote one state into another. See [Status and Evidence Lexicon](../01_SHARED_GOVERNANCE/007_STATUS_AND_EVIDENCE_LEXICON.txt#L4).

6. Infrastructure and Deployment Surface

There is no runtime deployment surface. The repository contains no application source, service manifest, package manager, test runner, container definition, CI workflow, or executable entry point.

The relevant infrastructure is artifact handling:

- The canonical source format is a plain-text tree with preserved imported source artifacts.
- The repository contains 453 `.txt` files, 21 Markdown contracts, 4 PDFs, 1 YAML orchestration file, 1 ZIP source archive, and hash metadata.
- The current source archive sidecar matches the ZIP content hash.
- The internal hash manifest resolves 478 entries; one source path has a filename-encoding mismatch even though its content hash matches.
- The workspace is not currently a Git repository, although the roadmap requires version control for reviewable batches, rollback, and continuity.
- Final packaging requires regenerated internal hashes, a ZIP retaining the `Thirsty's Codex` root, clean extraction, internal hash verification, and outer sidecar verification.

The canonical packaging sequence is defined in [the roadmap](../90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt#L380).

7. Extension and Continuation Patterns

Add or complete a lifecycle section

1. Read the roadmap and establish the exact baseline.
2. Read shared governance, upstream sections, adjacent sections, relevant role contracts, source register, and cross-reference register.
3. Define the field boundary and its handoff obligations.
4. Replace the visible placeholder with substantive content under the 24-part contract.
5. Run structural and terminology checks.
6. Integrate upstream and downstream relationships.
7. Perform independent challenge appropriate to consequence.
8. Update manifest, continuity, glossary, index, cross-references, roadmap, and checkpoint records.
9. Recompute hashes and record the batch evidence.

Add a role

Do not add or rewrite roles casually. The current role corpus is preserved source material. Any future role change requires an owner decision, source identity, authority analysis, expected-output contract, independence analysis, and reconciliation with the orchestration model.

Add a governance rule

Treat a new rule as a proposed change. Identify the authority, scope, affected roles and sections, enforcement path, evidence, exceptions, revocation, recovery, and owner ratification boundary before treating it as canonical.

Add a future AGI capability

The capability must be represented separately from authority. Define identity, role, assignment, tools, data boundaries, autonomy mode, approval points, monitoring, uncertainty, fallback, revocation, evidence, appeal, incident handling, and retirement. A more capable model increases the need for inspectability and bounded control; it does not remove human authority.

8. Rules and Anti-Patterns

Required behavior

- Preserve the human objective.
- Keep unknowns visible.
- Bound every material claim.
- Separate implementation from verification.
- Preserve dissent and failed attempts.
- Keep humans able to inspect, correct, pause, revoke, recover, and stop.
- Allow later evidence to reopen earlier work.

Prohibited shortcuts

- Treating file existence as completion.
- Treating a role title as authority.
- Treating documentation as runtime proof.
- Treating a successful demo as owner outcome proof.
- Treating a denial path as proof that the allow path works.
- Treating aggregate scoring as permission to violate mandatory constraints.
- Allowing the implementer to verify its own work without independent challenge.
- Using chat history as the only continuity record.
- Rewriting preserved source roles to make terminology look uniform.
- Treating draft V4 material as ratified universal authority.

9. Dependencies and Source Inputs

Source	Role in the Codex	Current qualification
Pasted text(20).txt	Full 406-field practical lifecycle list	Imported source input
Thirsty's Standard v3	Truthful scope, evidence, verification, continuity, and completion posture	Imported source input
Thirstys Agents 101	Twenty source-defined role contracts	Preserved without editorial rewriting
Thirstys Owner's Cockpit Standard	Human control and owner-operability concepts	Candidate source; not silently ratified
Thirstys Standard V4 Design Blueprint	Typed claims, evidence contracts, role assembly, recovery, dissent, trusted transitions	Draft and unratified design input
V4 Canonical Source Stack archive	Historical/source stack reference	Draft archive, hash-bound

Source inclusion does not make every source statement a finalized requirement. The authored Codex sections must reconcile and qualify source use.

10. Code Structure and Current Work State

```text
Thirsty's Codex/
|-- 00_FRONT_MATTER/          Purpose, lifecycle, navigation, glossary, index
|-- 01_SHARED_GOVERNANCE/     Human-AI, authority, evidence, continuity, cockpit, status
|-- 02_AGENT_ROLES/            Twenty roles, orchestration, expected outputs
|-- 03_SECTIONS/               406 lifecycle fields; 001-406 content-complete
|-- 90_REFERENCE/              Registers, checkpoint, roadmap, source identity
```

CODEX/docs/project-summary.md

```
|-- 99_SOURCE_MATERIAL/        Imported PDFs, text, ZIP, and sidecar inputs
|-- README.txt                 Canonical overview and continuation entry point
|-- MANIFEST.txt               Truthful section and assurance status counts
|-- CONTINUITY_MAP.txt         Durable state, limitations, and next safe action
|-- SHA256SUMS.txt             Internal artifact identities
`-- docs/                      Proposed operator pack and generated documentation
```

Current state and next action

All Sections 001-406 are CONTENT-COMPLETE. The historical checkpoint covers Sections 001-018, and the active assurance checkpoint is 019-060. The next safe action is to integrate and source-reconcile Sections 019-060, reopen only evidence-backed defects, update canonical support files, and preserve the exact boundary between content-complete, source-reconciled, hostile-reviewed, verified, and owner-accepted states.

The durable continuation plan is [90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt](../90_REFERENCE/006_REMAINING_SECTION_INT

Confidence Assessment

Claim area	Confidence	Basis
Repository purpose	High	README, introduction, title page, authored governance
File and status counts	High	Direct structural inventory and manifest comparison
Lifecycle architecture	High	406 filenames, lifecycle map, cross-reference register
Role boundaries	High	Preserved role files and expected-output contracts
Source authority	High for stated qualifications	Source register and section source-basis clauses
Final owner intent and ratification decisions	Unverified	No owner-acceptance record exists
External standard coverage	Unverified	The repository explicitly identifies reconciliation gaps
Git/version-control enforcement	Unverified and currently absent locally	No `.git` metadata found in this workspace

Key Local References

- [README.txt](../README.txt#L5)
- [MANIFEST.txt](../MANIFEST.txt#L5)
- [CONTINUITY_MAP.txt](../CONTINUITY_MAP.txt#L3)
- [Introduction](../00_FRONT_MATTER/002_INTRODUCTION.txt#L4)
- [How to Use the Codex](../00_FRONT_MATTER/003_HOW_TO_USE_THIS_CODEX.txt#L4)
- [Canonical Section Contract](../01_SHARED_GOVERNANCE/002_CANONICAL_SECTION_CONTRACT.txt#L4)
- [Authority, Scope, and Delegation](../01_SHARED_GOVERNANCE/003_AUTHORITY_SCOPE_AND_DELEGATION.txt#L1)
- [Evidence, Verification, and Truth](../01_SHARED_GOVERNANCE/004_EVIDENCE_VERIFICATION_AND_TRUTH.txt#L4)
- [Continuity, Handoff, and Recovery](../01_SHARED_GOVERNANCE/005_CONTINUITY_HANDOFF_AND_RECOVERY.txt#L4)
- [Owner Cockpit Requirements](../01_SHARED_GOVERNANCE/006_OWNER_COCKPIT_REQUIREMENTS.txt#L4)
- [Remaining Section Integration Roadmap](../90_REFERENCE/006_REMAINING_SECTION_INTEGRATION_ROADMAP.txt#L7)